



Unveiling the limitations of transformer models in time series forecasting

Alberto Miño Calero¹ · Adil Rasheed¹ · Anastasios M. Lekkas¹

Received: 24 April 2025 / Accepted: 23 March 2026
© The Author(s) 2026

Abstract

Transformer-based models are very popular in time series forecasting because of their ability to capture contextual information in natural language processing tasks. However, their application to time series has yielded mixed results. Performance improvements of Transformers in this domain are usually based on metrics such as MSE or MAE and have marginal percentage gains. We include statistical tests to examine the significance of such performance differences. Moreover, comparisons rarely account for the uncertainty that training such complex models entails. We investigate this by looking at the robustness in terms of training stability of Transformer-based models compared to simpler long-term time series forecasting linear (LTSF-Linear) models. Instead of focusing on adversarial or anomaly robustness, we measure it through standard deviation to common perturbations in data-driven algorithms, including initialized parameters and data splits. By analyzing model performance and robustness, and statistical significance, across various benchmark datasets, we find that Transformer-based models perform generally worse in our experiments, as pointed out by other works, and exhibit significantly higher variability of results, hence lacking robustness. This study highlights the challenges of using Transformer architectures for time series forecasting and underscores the importance of considering model robustness and stability, and the significance of the results, when choosing algorithms for time series forecasting.

Keywords Transformers · Time series forecasting · Deep Learning · LTSF-Linear models · Robustness · Training stability

1 Introduction

Time series analysis has long been a significant area of research [1], becoming essential in diverse fields such as climate science [2], healthcare [3], biology [4], and economics [5]. This evolution in time series forecasting has transitioned from simpler statistical models like autoregressive moving average (ARMA) [6] and autoregressive integrated moving average (ARIMA) [7] to more sophisticated models, such as support vector machines [8] and nonparametric Bayesian models [9]. Recently, the advent of Deep Learning has further

transformed the field, despite these models being computationally expensive and virtually uninterpretable without the use of specialized algorithms [10]. Their usefulness as universal approximators [11, 12] due to their numerous parameters, interconnectivity, and versatility has prompted significant advancements, albeit with some misuses [13, 14].

Various Deep Learning architectures have been employed with varying degrees of success, ranging from simple Multi-Layer Perceptrons [15] to more advanced Convolutional Neural Networks [16]. Networks designed for sequential data problems, such as Recurrent Neural Networks (RNN) [17] and Long-Short Term Memory (LSTM) networks [18], have also seen notable use. The introduction of Transformers [19] has revolutionized sequence modeling tasks [20–22], demonstrating their ability to model long-range dependencies. This innovation has led to the widespread application of Transformers in time series forecasting as well, resulting in a surge of Transformer-based solutions for time series analysis and forecasting, as reviewed by [23]. In the context of long-term time series forecasting, numerous Transformer-based models have recently been developed, including Informer

✉ Alberto Miño Calero
alberto.m.calero@ntnu.no
Adil Rasheed
adil.rasheed@ntnu.no
Anastasios M. Lekkas
anastasios.lekkas@ntnu.no

¹ Department of Engineering Cybernetics, NTNU, Faculty of Information Technology and Electrical Engineering, O. S. Bragstads Plass 2D, Trondheim 7034, Norway

[24], Autoformer [25], Etsformer [26], Crossformer [27], and FEDformer [28].

Each of these models brings unique innovations to time series forecasting. The Informer [24] model introduces a novel self-attention mechanism called ProbSparse, which is more efficient in terms of memory and time than the canonical self-attention [19], by focusing on dominant features to create more “concentrated” self-attention feature maps. The Autoformer [25] utilizes an autocorrelation mechanism to identify period-based dependencies, combined with a moving average filter to decompose trend and seasonal components. Crossformer [27] leverages cross-dimension dependency by organizing inputs into 2-D vector arrays, preserving time and dimensional relationships through dimension-segment-wise embedding. ETSformer [26] incorporates exponential smoothing into the attention mechanism and replaces the canonical self-attention with a frequency attention method, forming a model structured in modular decomposition blocks. FEDformer [28] employs a seasonal-trend decomposition method to capture general dynamics and applies a Fourier transform, based on the premise that most time series have sparse representations in well-known domains, thus utilizing the frequency domain instead of the time series domain.

The effectiveness of Transformers in time series forecasting is a topic of ongoing debate. In terms of percentual MSE improvements, the results of Informer [24] deliver a 9%, FEDformer [28] a 12%, and Crossformer [27] a 1% if not counting the ILI dataset, dropping to -1% (worse performance) when including it. Similarly, marginal percentages of 3% and 5% can also be observed in recent approaches, such as [29] and [30]. Moreover, none of them account for uncertainty, robustness, training stability, or convergence matters. While their contributions of introducing different components to the architectures of the models to deal with various challenges are positively valuable, the results lack the consideration of robustness or training stability to showcase how reliably such models converge to comparable optima, and how robust they are to learn from datasets in this domain. This is a known issue of complex models such as Transformers [31, 32] or, more generally, neural networks [33]. Additionally, there are no statistical tests to investigate the significance of the performance improvements claimed by the previously mentioned works.

Optimizing neural networks is challenging as they tend to fall into local optima, which, in turn, develop into issues such as bad performance, poor generalization capabilities, training instability, or convergence problems. Indeed, such issues can be linked to findings such as the existence of “Lottery Tickets” configurations, i.e., initial parameter configurations that have been found to propel specific after-training parameter configurations. This is studied in the model compression field, in works such as [34–36]. These ideas are usu-

ally exploited to sparsify models and reduce computational requirements while keeping performance from dropping. Although they fall out of the scope of this paper, this dependency on initial parameters is an interesting finding that may reinforce the necessity of evaluating the spread of training models under different parameter initializations and data splits. This type of robustness, training stability, or dependency of “lucky” initial parameters is essential to strengthen claims of certain models outperforming others, especially in settings such as the ones reproduced by this work, where computational requirements are not particularly prohibitive.

Another fair criticism is that these models are often compared in biased settings, with experiments made against other Transformer-based architectures designed for time-series forecasting, which may well be suffering from similar issues and not performing well if compared with different, simpler approaches, not based on Transformers or other Deep Learning approaches. This is the case in [24–28]. Only the occasional LSTM-based model, ARIMA, or Prophet [37] can be found in some of them, which, on the other hand, are known for issues such as accumulating errors and diminishing temporal correlation, suffering from heavy linear and stationary constraints [38, 39], and having problems with overfitting and underfitting trends, respectively. Delving deeper into the performance improvements claimed by some of the aforementioned works, [40] raises concerns about the benchmarks used in these comparisons, suggesting these biased comparisons create an artificially low bar for new Transformers to surpass. Additionally, certain properties of Transformers, such as permutation invariance and discrete space operation, which are advantageous in NLP tasks, might hinder their performance in time series forecasting due to the loss of positional information and the continuous nature of time series data.

In this work, we extend the experiments performed by [40] to investigate if Transformer-based models can compete in robustness with simpler approaches in addition to performance. The code is available in [GitHub](#), and the contributions of this paper are summarized as follows:

- Motivated by the proven lack of performance of Transformers in long-term time series forecasting (LTSF) tasks, we look deeper into the stability and robustness of four popular Transformer-based models and the LTSF-Linear to compare their suitability in this domain with nine benchmark datasets.
- The analysis of robustness in terms of stability is based on the assessment of the standard deviation in performance resulting from training multiple instances of each model, whose variations reside in different dataset splits and initialization of parameters.

- We perform a set of statistical tests to further look into the significance of the performance differences observed to support the standard deviation analysis.
- We find the Transformer-based models, in addition to displaying overall worse performance, have less robust training than the LTSF-Linear, thus revealing a struggle to consistently learn time dependencies needed and exhibiting a larger spread in performance. Overall, the analyzed Transformers show considerably less robust behavior than the LTSF-Linear despite these being much simpler, and despite the small perturbations, which are essentially different initial parameters and data splits, suggesting they may not be as well-suited for time series forecasting as most literature about Transformers in time series forecasting conveys.

The paper is structured as follows. Section 2 introduces the concepts needed to understand the document, starting with the models used for the experiments, the idea of robustness investigated, and the statistical validation process followed. Then, Section 3 explains the methodology, which includes the datasets used and the experimental settings. Section 4 contains the results of our experiments and the analysis of the findings, and Sect. 5 contains the conclusions of the study and future work.

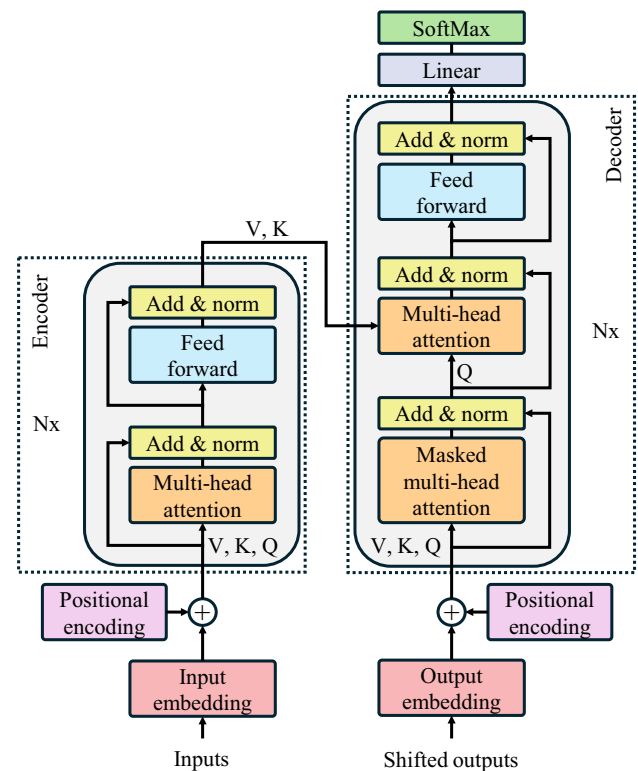
2 Theory

2.1 Transformers

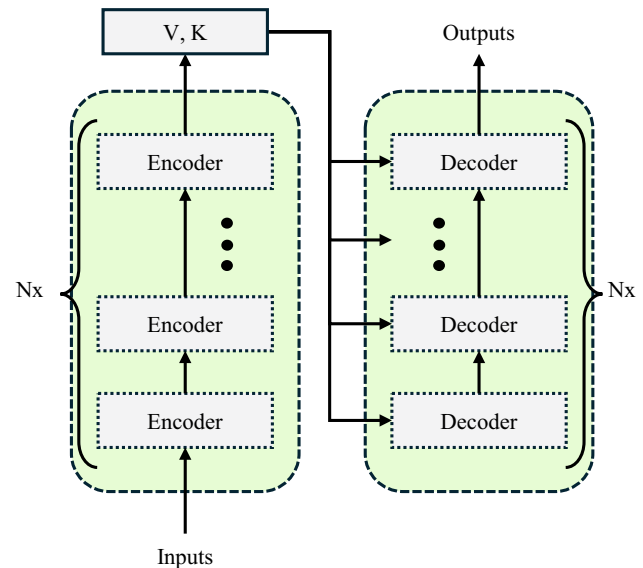
Transformers [19] are the next step of the Deep Learning community in the search for the best-performing model to solve sequential data tasks. Designed for NLP tasks, the great innovation of Transformers comes in the form of attention. Transformers integrate an attention mechanism that allows them to focus on the most important parts of the sequence they are fed. This poses a substantial improvement in NLP because they can deal exceptionally well with context information and better capture the meaning of and relationships between words. Unlike RNNs and LSTMs, Transformers are not inherently sensitive to sequence order without positional encodings, hence they are indifferent to word orders.

Transformers have an encoder-decoder stacked architecture that allows them to handle sequence-to-sequence tasks, removing the sequential components, which allows for a higher level of parallelization than RNNs and LSTMs. Both encoder and decoder internal structures are shown in Fig 1, where there are Nx stacks of each assembled as shown in Fig. 1b.

First, we have the input embedding, a transformation in which each element is turned into a numerical representation in the form of a vector of dimensionality h meant to capture



(a) Vanilla architecture of the Transformer [19].



(b) Stacked structure of the vanilla Transformer.

Fig. 1 Transformer architecture

the meaning of the words in NLP tasks. As such, it is a trainable parameter of the model. Since the embedding is fed to the model in a single time step, another element is needed to convey the information about the position of the items in the input sequence. That is the role of the positional encod-

ing, whose information is constant, does not change during training, and is set at the beginning.

With this information, the encoder can perform its role of processing input sequences into a latent space with a richer representation. The attention mechanism is formed by a multi-head attention layer followed by a normalization layer. Each attention head is characterized by three vectors named queries Q , keys K , and values V following Equation 1. The name self-attention is due to how it captures the relationships of each item with all the items of the sequence, and after being applied to the input embedding and positional encoding, the results of this layer contain information about meaning, position, and relationships between items of the sequence. Due to the nature of the operations, self-attention by itself is permutation invariant, does not require parameters, and can be adjusted in such a way that the results do not encode relations of items with other items that are ahead in the sequence, which can be relevant in applications such as NLP. The multi-head attention is a combination of self-attention heads together with trainable weights that allows the model to attend to information from different representation spaces at different positions. The normalization simply ensures that the values are kept in the $[0, 1]$ range.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{h}}\right)V \quad (1)$$

After the encoder is done, the decoder is responsible for generating the values we want to forecast in the predicted time horizon or sequence length. The output embedding works similarly to the input embedding, and there is again a positional encoding that is the same as the other one. The first element inside the decoder is a masked multi-head attention, which has an additional mask that allows the attention mechanism to avoid capturing the relationships of items of the sequence with other items of the future. Considering the results computed by the attention head as a squared matrix that relates every item to each other, similar to how a covariance matrix works, this can be done by masking the values above the main diagonal. The normalized output from this masked multi-head attention layer is used as Q to the next multi-head attention layer together with the outputs of the encoder, which are used as K and V . The rest of the layers work as described. At the end, there is a simple feed-forward layer followed by a softmax that outputs the probabilities. Note that these probabilities are assigned to the tokens learned by our model, the vocabulary in NLP tasks, and hence the output values are discrete instead of continuous.

Regarding their training, since Transformers avoid the sequential processing of time steps altogether thanks to the positional encodings, they can use the traditional back-propagation algorithm of gradient descent [41], allowing

better parallelization and relatively fast training given their typical size.

The Transformer-based models used for this work contain an additional characteristic to overcome the limitation of vanilla Transformers, which cannot preserve positional information that might be relevant for time series forecasting. This approach consists of providing the model with temporal information about the time series order of the sequences [24], relevant through the embeddings, in addition to the vanilla positional encoding and input embeddings. This temporal information is passed in the form of timestamps that enclose information about the global position in the time dimension and are handled by Transformer-based models as another embedding with learnable parameters [42]. The particularities of each Transformer are described in the following sections.

2.1.1 Informer

Vanilla Transformers suffer from several issues, including the time and memory requirements of their self-attention strategy, and the error accumulation due to the stack of autoregressive decoders of their architecture, which suffers from handling long time-series sequences. These challenges are the motivation behind the Informer solution [24], which introduces modifications to the self-attention mechanism, the encoder, and the decoder with LTSF-Linear tasks in focus. Based on the low-rank property of the self-attention matrix that Transformers use, Informer proposes a novel self-attention mechanism called ProbSparse, a distilling operation that decreases time and memory complexity, and a modification to the decoder to avoid step-by-step predictions and change the inference process from step-by-step to one forward operation, improving performance on long-time sequences.

ProbSparse affects the query matrix Q on the vanilla attention mechanism from Equation 1. The new Q' is sparsified through the Query Sparsity Measurement M . This metric uses Kullback-Leibler divergence to measure the likeness between the distribution of each row of the query matrix Q and the uniform distribution. What makes a difference between these two is the presence of pairs of Q and K values during the dot product operation of the attention mechanism that dominate the resulting values, which pushes the distribution away from the uniform distribution. By using these ideas and an empirical approximation for the efficient acquisition of M , the attention mechanism takes in a sparse Q' matrix that includes only the most important queries, effectively reducing the number of operations to $\mathcal{O}(L \ln L_Q)$, with L being the input length.

Due to the modification introduced by ProbSparse, the encoder takes redundant combinations of V , hence the distilling operation for the self-attention mechanism. Working

over the multi-head attention, this operation uses a 1-D convolutional layer on the time dimension with the $ELU(\cdot)$ activation function [43] and a max-pooling layer to trim the time dimension of the input and focus on the dominating features.

The final modification introduced by Informer is made to the decoder. It still has the same structure as Transformers, but it performs a generative inference procedure. This new process takes long sequences of the input sequence, like the earlier slice before the output sequence, and performs the inference over all these values instead of just taking in specific flags as tokens.

2.1.2 Autoformer

The novelty introduced by the Autoformer [25] lies in the application of a seasonal-trend decomposition process to each neural block, creating a Transformer-based decomposition forecasting architecture. This architecture enables Autoformer to model the data as a combination of trend and seasonal components, in addition to the remaining elements and patterns of the data.

The decomposition architecture is achieved by limiting the decomposition to an inner block of the Autoformer model. The decomposition blocks replace the addition and normalization layers from Fig 1 in encoders and decoders. The series decomposition blocks apply a moving average window that smooths out periodic fluctuations at the same time that extracts trends discovered in the long-term relationships. The moving average window is done by using the $AvgPool(\cdot)$ function with padding to keep the length of the series constant.

Given the decomposition of seasonal and trend-cyclical components performed by the encoder, the input of the decoder contains the decomposition of the second half of the input of the encoder, providing recent information and placeholders of length equal to the length of the prediction, filled with scalar values.

Another modification made to the encoder and the decoder is the inclusion of an autocorrelation mechanism with series-wise connections that substitutes each multi-head attention mechanism from Fig 1. The goal of this new mechanism is to improve the utilization of information within the model by discovering period-based dependencies using the series autocorrelation time delay aggregation to aggregate similar sub-series.

The calculation of period-based dependencies is based on the stochastic process theory for real discrete-time processes and uses the Fast Fourier Transform to reflect time-delay similarities between the original and its lagged series. On the other hand, the time delay aggregation aligns sub-series of equal phase positions within an estimated period, differing from the point-wise dot-product aggregation opera-

tion generally performed by self-attention mechanisms, and aggregating the resulting elements by softmax-normalized confidences. Autoformer also achieves similar complexity to Informer, with $\mathcal{O}(L \ln L)$ for series sequences of length L .

2.1.3 FEDformer

Frequency Enhanced Decomposed Transformer, referred to as FEDformer in the literature, was introduced by [28] to take advantage of the fact that time series data tend to have sparse representation via the Fourier transform. Similar to Autoformer, FEDformer also incorporates a decomposition method for seasonalities and trends, aiming to capture the time series' global structure. The main novelties of FEDformer are the frequency-enhanced blocks (via wavelets or Fourier), the frequency-enhanced attention mechanism, and randomly selected Fourier components for the sparse representation of the time series. Similarly to Autoformer and Informer, they achieve $\mathcal{O}(L \ln L)$ complexity by using Fast Fourier transform, further improved by taking a random subset of the Fourier basis with the scale bounded by a scalar. As pointed out by [40], the FEDformer-f, which uses Fourier-enhanced blocks instead of wavelets, has better performance than FEDformer-w. Hence, we will focus on this version of the model.

FEDformer is a multilayer network where each encoder layer is composed of frequency-enhanced blocks and mixture-of-experts decomposition blocks, and each decoder has the same, but with the addition of a frequency-enhanced attention layer, followed by more mixture-of-experts decomposition blocks. This structure decomposes the prediction into seasonal and trend components. The final prediction is formed by projecting the deep seasonal component to the output dimension and adding the trend component.

Regarding the specifics of the FEDformer components, the frequency-enhanced blocks of FEDformer-f, present in both encoder and decoder, use Fourier transforms to convert a linearly projected input to the frequency domain, where only some M randomly selected modes are kept. Then, the layer uses a parametrized and randomly initialized kernel in combination with the Fourier transforms and a padding operation to reverse the domain from frequency to time. The frequency-enhanced attention that is located in the decoder layers converts the canonical expression of Transformers, queries, keys, and values from Equation 1 to the frequency domain again with the Fourier transform, hence performing the attention mechanism over this transformed domain.

The mixture-of-experts decomposition blocks acts as a countermeasure element to address the difficulty of learning complex seasonal and trend patterns with only a fixed window average pooling. To solve this problem, this block contains several average pooling filters with different sizes,

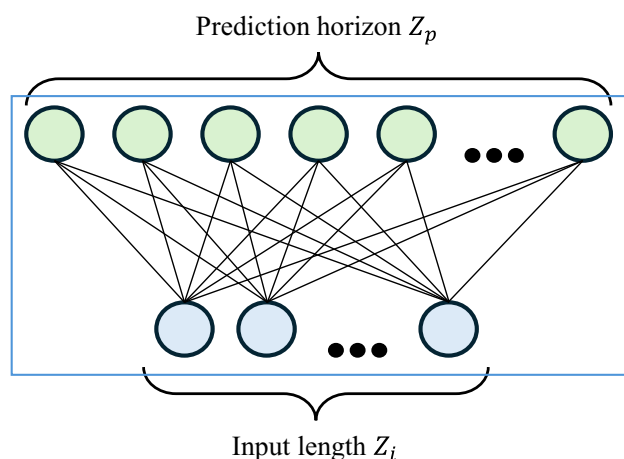


Fig. 2 Base architecture of the models from the LTSF-Linear family. The fully connected scheme applies to the temporal dimension but not to the features, which are modelled in independent channels.

allowing the model to learn multiple trend components that can be combined to form more complex trends in the data.

2.2 LTSF Models

The LTSF-Linear models introduced in [40] are a family of three models designed to make a baseline comparison between one-layer neural networks with linear activation functions and Transformer-based models in the LTSF task. One novelty introduced by these models is that they perform multi-step forecasting instead of rolling forecasting. The advantage is that by forecasting all the values of the desired prediction horizon, they avoid the accumulation of error that typically happens in rolling forecasting strategies, where each next value uses the previous output value as input, which contains a certain level of error. The second key aspect is that they are autoregressive, as they model temporal but not spatial correlation between features, contributing to further reducing their complexity.

There are three LTSF-Linear models built upon a one-layer linear neural network. The first and simplest architecture is the proper neural network without hidden layers and linear activation functions. A schematic of the architecture can be seen in Fig. 2. The model uses as many neurons as the input sequence length Z_i , and as many outputs as the values it has to predict Z_p . Conceptually, this model is just a regression over the historical values with a weighted sum operation.

The other two models have preprocessing methods added to the basic Linear model. First, the NLinear model is a modification designed to improve the performance of the Linear model when facing skewed distributions. This method consists of subtracting the input from the last value of the sequence before the linear layer. After it has gone through

the linear layer, the subtracted part is re-added to the output before the prediction. This subtraction and addition essentially serve as a normalization technique for the input sequence.

The second improvement is the DLinear model. This model integrates a decomposition scheme inspired by the Autoformer [25] algorithm into the Linear model. First, it decomposes the input into a trend component using a moving average kernel and a seasonal component. Then, each component is processed through two one-layer linear layers, and the resulting features are aggregated to make the final prediction. This treatment of the trend greatly improves the DLinear performance with respect to the Linear model, particularly in scenarios where the data exhibits a noticeable trend.

2.3 Robustness and stability

As model complexity increases, the optimization process becomes more difficult because of the presence of multiple local minima [44]. Based on the many factors that characterize Deep Learning solutions, such as parameter initialization and hyperparameter optimization, i.e., learning rates, batch sizes, regularization techniques, etc, this can result in entirely different trained models, leading to large model uncertainty that makes them unfit for predictions.

Robustness is generally understood as the capability of a system to resist perturbations such as stress, pressure, or any unpredictable changes without losing functionality, which in the Deep Learning context, can be linked to models falling into poor local minima. Despite this general definition, robustness is most often studied against adversarial attacks, as can be gathered from searching the literature [45–47]. Furthermore, there seems to be some evidence from the literature, such as [48, 49], pointing out how this important concept in Machine and Deep Learning is sometimes used without clearly specifying its meaning. This serves as motivation for clearly specifying the kind of robustness studied in our work.

Adhering to the broad definition of robustness, it can be analyzed with respect to any type of perturbations, allowing answering questions other than how resilient models are to adversarial attacks. For instance, robustness analysis can be performed to study input rates in spiking neural networks [50]. In [51], robustness is examined against the presence of noisy labels, and in [52] it is done with stain variations, i.e., a particular type of distribution shift. To analyze another aspect of how fitting Transformer-based models are for the LTSF task, we examine how stable these models are in the learning process, hence revisiting the traditional concept of training stability.

Training stability refers to how small changes in the training data affect the performance and generalization capa-

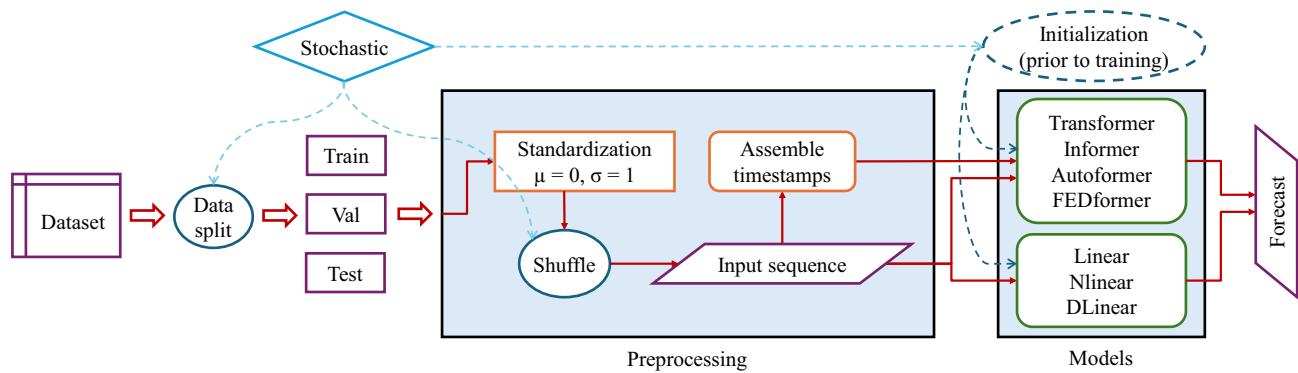


Fig. 3 General pipeline diagram of the Transformer and LTSF-Linear models. The stochastic processes that our approach exploits to study the robustness are marked in cyan. Preprocessing refers to common

data processing steps for Transformer-based and Linear models. Note that the timestamps are only passed to the Transformers

bilities of models, such as in [53]. When training stability is flawed, models can be predisposed to suffer from performance issues, which, in terms of the optimization problem, can be understood as models falling into a plethora of local minima. This, in turn, may point to fundamental issues with the type of models used for a particular type of task, such as the one we are examining here.

Henceforth, our approach is to analyze the robustness of Transformer-based models and the LTSF-Linear models by looking into the training stability of the models. The randomness of learning algorithms typically comes from sampling data to create the training, validation, and testing datasets [54], as well as initialization of the parameters in neural networks, as pointed out in Fig. 3. Under the assumption that the available data is representative enough to train a learning algorithm, small changes in these processes should not lead to big performance differences. Otherwise, we would need to rely on getting “lucky” with our training to get a robust model out of our learning algorithm. The analysis of this work takes into account this randomness by training multiple models under different settings with respect to the data splitting into train/validation/test subsets and the initialization of the networks’ parameters. We then use standard deviation to gather and analyze the results, examining how stable our two families of models are in their training to learn sufficiently proper time dependencies from a set of benchmark time series to make forecasts.

2.4 Statistical validation

This study is reinforced by traditional techniques to check statistical significance in the performance differences based on [55]. The statistical study consists of the Friedman test with a post-hoc Nemenyi test, whose results are incorporated into the analysis as statistical verification.

The Friedman test [56, 57] is a non-parametric method to detect differences between experimental units under observation and the conditions of study. In the medicine domain, these typically correspond to patients and treatments, respectively [58, 59]. However, the test can also be used to study if there is a significant difference between the performance of several models across datasets [55].

The Friedman test is based on the ranks assigned to the models for each dataset individually, where the best performing receives rank 1, the second best rank 2, and so on. Then, the test compares the average ranks of each model. In practice, however, the equivalent Equation 2 from [56] facilitates the computation by using rank sums instead of averages, where d refers to the number of datasets, m to the number of models compared, and r_{ij} to the rank of the i -th model in the j -th dataset.

$$\chi_F^2 = \frac{12}{dm(m+1)} \sum_{i=1}^m \left(\sum_{j=1}^d r_{ij} \right)^2 - 3d(m+1) \quad (2)$$

In the Friedman test, the null hypothesis states that all models perform equivalently, meaning their ranks are sampled from the same distribution. The test statistic χ_F^2 is compared against the critical value of the Chi-Square with $df = m - 1$ degrees of freedom at a chosen significance level α . When χ_F^2 is greater than this critical value, the null hypothesis is rejected, which indicates that at least one model performs significantly differently from the others. The Chi-square approximation is considered reliable when the number of datasets is reasonably large, typically $d \geq 10$ and $m \geq 5$ [55], and the Iman Davenport [60] correction is not needed. A rejection of the null hypothesis typically motivates a post-hoc test to identify which models differ significantly.

A post-hoc analysis based on the Nemenyi test [61] is applied afterward, with the null hypothesis that the models

Table 1 Statistics of the benchmarking datasets, with different time resolutions and total time steps. In real time, prediction horizons can vary from less than a day to more than a year

Datasets	Traffic	Electricity	Exchange-Rate	Weather	ETTh	ETTm	ILI
Features	862	321	8	21	7	7	7
Time steps	17544	26304	7588	52696	17420	69680	966
Time resolution	1 hour	1 hour	1 day	10 min	1 hour	15 min	1 week
Pred. horizons (Z_p)	{96, 192, 336, 720}						{24, 36, 48, 60}
$Z_p = 96$ in hours	96	96	2304	16	96	24	($Z_p = 24$) 4032
$Z_p = 720$ in hours	720	720	17280	120	720	180	($Z_p = 60$) 10080

perform equally well. The null hypothesis can be rejected when the average ranks of two models have a difference of at least the CD following Equation 3, where q_α is the studentized range statistic at a significance level α , divided by $\sqrt{2}$.

$$CD = q_\alpha \sqrt{\frac{m(m+1)}{6d}} \quad (3)$$

This test can be used when the Friedman test rejects the null hypothesis to make pairwise comparisons between models and find significant differences, indicating which of the models are significantly different in terms of performance.

3 Methodology

3.1 Data

To extend the existing comparative studies of [40], we reuse the same nine benchmark datasets. These datasets are multivariate time series that contain information on a variety of domains: Traffic, Electricity, Weather, ILI, Exchange Rate [62], and Electric Transformer Temperature (ETT) [24] (ETTh1, ETTh2, ETTm1, and ETTm2). The data splitting is shared by all datasets and models, consisting of subsets of 70%, 10%, and 20% of the total samples for training, validation, and testing, respectively. The statistics of the dataset are displayed in Table 1, and a brief description of each is as follows:

- Traffic [63]: collects data from individual detectors across metropolitan areas of the State of California, recorded from sensors located on San Francisco freeways from 2015 to 2016.
- Electricity [64]: electricity consumption dataset in kWh of 321 clients recorded from 2012 to 2014.
- Exchange-Rate [65]: collects the daily exchange rates of eight countries from 1990 to 2016.
- Weather [66]: contains data from several weather indicators in Germany from 2020.

- ILI [67]: contains the ratio of patients with influenza-like illness and the number of patients with influenza.
- ETT [24]: contains information about crucial indicators in the long-term electric power deployment with oil and load features of electricity transformer temperatures from July 2016 to July 2018 from two counties in China. It is divided into 4 usable datasets:

- ETTh: has a time resolution of 1 hour for up to 17420 samples. ETTh₁ contains samples from the first year, while ETTh₂ has samples from the second year.
- ETTm: has a time resolution of 15 minutes for a total of 69680 samples, ETTm₁ with the data from the first year, and ETTm₂ from the second year.

The datasets present a combination of usual time-series forecasting challenges, such as periodicities in ETT, Weather, and ILI, trends in ILI, and a combination of stationary, trend-stationary, and non-stationary features (generally present in every dataset) according to augmented Dickey-Fuller and KPSS tests. The datasets also have different sizes (total length) and numbers of features, time resolutions, and skewed distributions in some features. Furthermore, irregularities can be observed in some of them, such as in ETTh₂, Electricity, and Exchange rate, where certain sequences, i.e., short temporal periods, exhibit behaviors different than the rest of the dataset. These datasets are handled as multivariate time series; therefore, the models must predict the evolution of all the features.

3.2 Experimental settings

To configure the hyperparameters of the models, the starting point was the default configuration from [40], also considering the initial setup of the Transformer-based models from their canonical works [24, 25, 28]. Given the underperforming results in comparison with the literature references obtained, we conducted a manual tuning investigating hyperparameters that do not modify the models' sizes, and looked into batch sizes, learning rates, training epochs, and early stopping patience hyperparameters, which proved to be the

Table 2 Hyperparameter values explored. In bold are the best values for each family of models. Learning rate and batch size optimized values depend on the dataset for the LTSF-Linear models

Hyperparameters	LTSF-Linear	Transformers
Learning rate	0.05, 0.01 0.005, 0.001 0.0005, 0.0001	0.0005, 0.0001
Batch size	8, 16, 32, 64	16, 32 , 64
Epochs	10, 20 , 30	1, 2, 3, 10
Patience	2, 3, 5	2, 3 , 5

most impactful on the performance of the models in preliminary tests. With these, comparable metrics to those found in the literature were reached, only slightly underperforming for some LTSF-Linear cases.

The hyperparameter tuning process is summarized in Table 2, which shows the values tested, with the best values marked in bold. Note that we did not perform a grid search, and therefore, not all values were tested for all models and datasets; we experimented with different values in order to achieve equivalent results to the literature. For the Transformers, increasing the epochs was sufficient to achieve similar performance, and even better in some cases. Other hyperparameters did not yield significant improvements, and the default values were used. For the LTSF-Linear models, an increase in epochs and early stopping patience improved the results, but fine-tuning of the learning rate and batch sizes was also required to achieve comparable performance for several datasets. For that reason, the specific configurations regarding these two hyperparameters must be checked in the code.

Some other relevant shared hyperparameters are as follows. The input length varied depending on the type of model. For the LTSF-Linear models, long input sequences achieved the best results, and a value of $Z_i = 336$ was the best performing, while Transformer-based models did not improve or got worse with values higher than $Z_i = 96$. The analyzed prediction horizons cover short and long-term time dependencies and are $Z_p = \{96, 192, 336, 720\}$ for every dataset except for ILI, which, due to its smaller size, needs shorter prediction horizons $Z_p = \{24, 36, 48, 60\}$. Note that these prediction horizons do not account for the time resolution of each dataset and instead refer exclusively to the number of time steps the models are trained to predict. Therefore, the actual time horizon depends on the resolution of each dataset. Table 1 includes the conversion to hours for the shortest and longest prediction horizons for each dataset for reference. Regarding the rest of the hyperparameters, such as learning rates, batch sizes, or early stopping, we refer the reader to the code for specific details of each model configuration. The LTSF-Linear models were trained on a CPU Intel i9-12900K

due to the chance of parallelizing training across multiple cores, which sped up training intensely, while Transformer-based models needed much more computational resources, making parallelization not possible; hence, a GPU NVIDIA RTX 3090 was used.

We chose an approach based on random seeds to handle the training randomness to analyze the robustness of these algorithms. The randomness typically comes from the two processes of splitting the data and initializing the parameters of the network. Both processes are typically transparent to the researcher, aside from picking split percentages and an initialization algorithm that fits the data and the model, and both depend on the random number generators of the libraries and functions that execute them. These libraries have a default “stochasticity” where the seed is transparent, which is desirable in certain cases in Machine Learning, but that hinders reproducibility and, if not handled, can lead to “lucky” model scenarios. This problem happens when a certain model achieves remarkably good results on sparse occasions, while, generally, its performance is significantly worse. Our approach consists of using the deterministic mode of the libraries involved in the training of the models to control the training stochasticity and enhance the reproducibility of the results, and feeding these libraries with several random seeds S to train S models for each configuration. By doing this, we get S models with the same hyperparameters trained over different data splits and initialization parameters that allow us to measure their robustness and training stability, as shown previously by Fig. 3. Note that the figure only displays common data preprocessing steps, but each model performs its own processing steps, decomposing the time series to improve their capabilities. These steps are, however, not really within the data preprocessing, as they already involve models’ trainable parameters.

3.2.1 Metrics

This work analyzes the performance in terms of Mean Squared Error (MSE). Given the multivariate forecasting task and multiple experiments for each analyzed case, the aggregated MSE (MSE_{agg}) used is a simple average of the MSE of each of the $N_{exp} = 10$ experiments, as Equation 4 shows, performed for each given type of model, dataset, and prediction horizon (MSE_{exp}). We selected 10 independent training runs as a compromise between computational feasibility and statistical representativeness. Each MSE_{exp} is computed following Equation 5, where y refers to the prediction value, $\hat{y}$ to the true value, N_f to the total number of features predicted, and N_p to the total number of predictions (i.e., number of test sequences). The subscripts i , j , and k express feature, prediction, and experiment, respectively, in both expressions.

$$MSE_{agg} = \frac{\sum_{k=1}^{N_{exp}} MSE_{expk}}{N_{exp}} \quad (4)$$

$$MSE_{exp} = \frac{\sum_{j=1}^{N_p} \sum_{i=1}^{N_f} (y_{i,j} - \hat{y}_{i,j})^2}{N_f N_p} \quad (5)$$

Second and more significantly for this work, we examine the standard deviation σ as a statistic of spread as Equation 6 shows, where $\mu = \sum_{l=1}^{N_{exp}} MSE_{expl} / N_{exp}$, i.e., the mean of the MSE of the aggregated experiments. Based on this statistic, we analyze the robustness of our models based on the aspects we defined earlier in Sec. 2.3.

$$\sigma_{agg} = \sqrt{\frac{\sum_{k=1}^{N_{exp}} (MSE_{expk} - \mu)^2}{N_{exp}}} \quad (6)$$

Regarding the Friedman test, in [55], the ranks depend on the models' performance, taking the average of repeated trainings. Thus, the approach has the same liability as comparing performance based solely on MSE metrics. There is no account for training stability, and models that are more robust to training stochasticity do not have this advantage taken into account.

To address this issue, we apply the Friedman test in two complementary ways. In the global analysis, we follow the conventional approach where each dataset prediction horizon combination represents one experimental condition. The mean MSE across the 10 independent training instances of each model is used as a single representative performance value, yielding one rank per model and dataset Z_p pair, which are then used for the test. These ranks are then averaged across all $d = 36$ conditions to obtain a single global average rank per model.

In contrast, to analyze robustness and training stability, we also apply the Friedman test independently to each dataset- Z_p pair. In this configuration, the training instances themselves (each corresponding to a distinct random seed) are treated as the experimental conditions ($d = 10$). Thus, the ranks are assigned based on the performance of each individual training run rather than on averaged MSE values. This allows us to test whether models differ significantly in how stable their training is under stochastic perturbations, enabling the Friedman test to naturally account for robustness, and complement the σ_{agg} analysis by evaluating stability through rank-based significance rather than dispersion alone.

3.2.2 Models compared

We include four Transformer-based methods, one vanilla Transformer [19] and three more recent approaches designed for time series, which are Autoformer [25], Informer [24],

and FEDformer [28] models, as described in Sec. 2.1. These models are among the most advanced methods that make use of Transformers for time series forecasting, but they still show easy-to-prove issues with learning time dependencies. As for the LTSF-Linear models introduced in [40], they are described in Sec. 2.2. For all the models and experiments, the data is subjected to a preprocessing standardization step where the mean is removed and the variance is scaled to one. Furthermore, each model, aside from the vanilla Transformer and the Linear ones, executes built-in data processing operations. The LTSF-linear models demonstrate how three simple methods can easily outperform Transformers in a set of nine time series datasets with different characteristics, both in terms of performance, as pointed out by [40], and stability and robustness, as shown by the results of our experiments.

4 Results and discussions

Table 3 shows the results in terms of the two chosen performance metrics of the extensive evaluation conducted. As can be seen, the LTSF-Linear models perform better for every dataset, with improvements ranging up to 40%, with an average improvement of 21%. Following the results presented in [40], our metrics show little deviation in comparison in terms of MSE performance. Despite the relative simplicity of the LTSF-Linear models, they seem to deal with the dataset complexities better on almost every occasion, managing non-stationary characteristics such as trends (ILI) and seasonalities (ETT, weather), and other complications such as distribution shifts, which are more present in small datasets such as ILI, ETTh1, and ETTh2. FEDformer is able to achieve similar performance in the ETTh1 dataset, and the reason for this can be found in [40]. In our results, FEDformer is the only Transformer-based model that, for one dataset (ETTh1) and one prediction horizon ($Z_p = 96$), marginally outperforms the best LTSF-Linear model by 0.82%, and although it falls behind for longer prediction horizons, the difference is smaller than in the original [40] work.

Beyond raw performance, we verified the statistical significance of these differences using the Friedman and Nemenyi tests to ensure that observed differences were not due to chance. To do so, we perform a statistical analysis divided into three steps. First, the Friedman test, as explained in Sect. 2.4 applied to all the models and datasets, with the Nemenyi post-hoc analysis. In this step, we consider each of the datasets in combination with each Z_p , arriving at a total of $d = 36$ effective experiments to be considered in the test. Since we train 10 independent instances of each model, we use the mean MSE to obtain a single representative performance score per model that can be used for the ranking for each dataset and Z_p . For $\alpha = 0.05$ and $df = 6$, the critical value is 12.592. The resulting $\chi_F^2 = 173.49$

Table 3 Average performance of 10 models, where the best results are marked in green, best of each family in bold, percentage of improvement under Imp. column, and rank in parentheses. Red values indicate where the improvement percentage is in favor of Transformers. note the Performance differences to [40] on ILI, ETTh2, and ETTm2 datasets, for

which the LTSF-Models underperform. LTSF-Linear outperforms the Transformers in every occasion except for ETTh1 for ($Z_p = 96$), where FEDformer surpasses DLinear by a small margin. ETTh1 is the only case where the Linear models do not consistently outperform Transformers

Model		Linear	NLinear	DLinear	Imp.	Transformer	Autoformer	Informer	FEDformer
Metric		MSE	MSE	MSE	% MSE	MSE	MSE	MSE	MSE
Electricity	96	0.13373 (2.0)	0.13452 (3.0)	0.13371 (1.0)	28.40%	0.25783 (6.0)	0.19917 (5.0)	0.33945 (7.0)	0.18676 (4.0)
	192	0.14907 (2.0)	0.15068 (3.0)	0.14906 (1.0)	28.14%	0.26928 (6.0)	0.22077 (5.0)	0.34581 (7.0)	0.20743 (4.0)
	336	0.16499 (2.0)	0.16772 (3.0)	0.16498 (1.0)	22.92%	0.27696 (6.0)	0.24689 (5.0)	0.36710 (7.0)	0.21404 (4.0)
	720	0.20151 (2.0)	0.20645 (3.0)	0.20144 (1.0)	18.61%	0.29276 (6.0)	0.28289 (5.0)	0.39037 (7.0)	0.24750 (4.0)
Exchange	96	0.10056 (3.0)	0.09463 (2.0)	0.09190 (1.0)	34.41%	0.74842 (6.0)	0.15111 (5.0)	0.95089 (7.0)	0.14010 (4.0)
	192	0.19588 (2.0)	0.19566 (1.0)	0.19736 (3.0)	28.23%	1.21690 (7.0)	0.36075 (5.0)	1.12054 (6.0)	0.27261 (4.0)
	336	0.34402 (2.0)	0.34998 (3.0)	0.34076 (1.0)	24.91%	1.67196 (7.0)	0.50071 (5.0)	1.56820 (6.0)	0.45381 (4.0)
	720	0.93832 (1.0)	0.98189 (3.0)	0.94171 (2.0)	19.49%	2.81934 (7.0)	1.24058 (5.0)	2.68098 (6.0)	1.16542 (4.0)
Traffic	96	0.42578 (2.0)	0.42308 (1.0)	0.42614 (3.0)	26.68%	0.65175 (6.0)	0.62912 (5.0)	0.72967 (7.0)	0.57702 (4.0)
	192	0.43868 (2.0)	0.43593 (1.0)	0.43894 (3.0)	27.98%	0.66672 (6.0)	0.64372 (5.0)	0.74799 (7.0)	0.60531 (4.0)
	336	0.45230 (2.0)	0.44847 (1.0)	0.45262 (3.0)	28.18%	0.66314 (6.0)	0.62440 (4.0)	0.82766 (7.0)	0.62629 (5.0)
	720	0.48279 (2.0)	0.47689 (1.0)	0.48308 (3.0)	24.58%	0.68467 (6.0)	0.65564 (5.0)	0.99804 (7.0)	0.63229 (4.0)
Weather	96	0.14535 (3.0)	0.14359 (1.0)	0.14437 (2.0)	39.14%	0.39176 (7.0)	0.26319 (5.0)	0.38505 (6.0)	0.23593 (4.0)
	192	0.18807 (3.0)	0.18665 (1.0)	0.18693 (2.0)	33.10%	0.59354 (7.0)	0.30522 (5.0)	0.45239 (6.0)	0.27901 (4.0)
	336	0.24142 (3.0)	0.23935 (1.0)	0.24047 (2.0)	33.76%	0.72035 (7.0)	0.36442 (5.0)	0.55290 (6.0)	0.36136 (4.0)
	720	0.31770 (1.0)	0.31834 (3.0)	0.31782 (2.0)	20.93%	0.99044 (6.0)	0.42367 (5.0)	1.00264 (7.0)	0.40177 (4.0)
ETTh1	96	0.37954 (4.0)	0.37929 (3.0)	0.37780 (2.0)	0.82%	0.94121 (6.0)	0.45079 (5.0)	0.94138 (7.0)	0.37471 (1.0)
	192	0.41625 (2.0)	0.41527 (1.0)	0.41734 (3.0)	0.69%	0.94635 (6.0)	0.48786 (5.0)	0.96179 (7.0)	0.41814 (4.0)
	336	0.44618 (2.0)	0.44151 (1.0)	0.45022 (3.0)	2.00%	1.05378 (6.0)	0.51953 (5.0)	1.11114 (7.0)	0.45053 (4.0)
	720	0.48886 (4.0)	0.45301 (1.0)	0.48761 (3.0)	5.92%	1.07480 (6.0)	0.52081 (5.0)	1.18650 (7.0)	0.48153 (2.0)
ETTh2	96	0.38634 (5.0)	0.28906 (1.0)	0.38262 (4.0)	15.49%	2.26771 (6.0)	0.37261 (3.0)	3.28200 (7.0)	0.34204 (2.0)
	192	0.54625 (5.0)	0.34797 (1.0)	0.52558 (4.0)	18.60%	4.54673 (6.0)	0.45576 (3.0)	5.99005 (7.0)	0.42748 (2.0)
	336	0.67938 (5.0)	0.37595 (1.0)	0.65960 (4.0)	18.20%	4.13687 (6.0)	0.47081 (3.0)	4.85042 (7.0)	0.45957 (2.0)
	720	0.97952 (5.0)	0.42025 (1.0)	0.97363 (4.0)	8.39%	3.03423 (6.0)	0.47704 (3.0)	3.96340 (7.0)	0.45876 (2.0)
ETTh1	96	0.28622 (1.0)	0.29075 (3.0)	0.28642 (2.0)	20.07%	0.58586 (6.0)	0.47597 (5.0)	0.64360 (7.0)	0.35808 (4.0)
	192	0.32736 (2.0)	0.33116 (3.0)	0.32705 (1.0)	18.20%	0.74469 (6.0)	0.58029 (5.0)	0.76470 (7.0)	0.39983 (4.0)
	336	0.36686 (1.0)	0.37049 (3.0)	0.36701 (2.0)	16.43%	1.00790 (6.0)	0.68094 (5.0)	1.12365 (7.0)	0.43901 (4.0)
	720	0.42800 (2.0)	0.42910 (3.0)	0.42679 (1.0)	12.48%	1.08782 (6.0)	0.64191 (5.0)	1.10694 (7.0)	0.48764 (4.0)
ETTh2	96	0.18978 (3.0)	0.16296 (1.0)	0.18745 (2.0)	14.15%	0.46129 (6.0)	0.28539 (5.0)	0.48165 (7.0)	0.18981 (4.0)
	192	0.30151 (5.0)	0.21832 (1.0)	0.28533 (4.0)	15.80%	0.92149 (7.0)	0.28406 (3.0)	0.77206 (6.0)	0.25928 (2.0)
	336	0.41035 (4.0)	0.27354 (1.0)	0.43961 (5.0)	14.94%	1.26497 (6.0)	0.34650 (3.0)	1.39852 (7.0)	0.32157 (2.0)
	720	0.69295 (4.0)	0.37047 (1.0)	0.72224 (5.0)	12.97%	3.15135 (6.0)	0.43785 (3.0)	3.36672 (7.0)	0.42569 (2.0)
ILI	24	2.22816 (3.0)	2.00212 (1.0)	2.21013 (2.0)	38.64%	4.63107 (6.0)	3.54746 (5.0)	5.33868 (7.0)	3.26287 (4.0)
	36	2.24992 (3.0)	1.93162 (1.0)	2.24062 (2.0)	27.04%	4.67947 (6.0)	3.36260 (5.0)	5.16440 (7.0)	2.64745 (4.0)
	48	2.30571 (2.0)	1.91329 (1.0)	2.31174 (3.0)	26.94%	4.82786 (6.0)	3.14211 (5.0)	5.14257 (7.0)	2.61875 (4.0)
	60	2.44082 (3.0)	1.93573 (1.0)	2.43887 (2.0)	31.56%	5.20437 (6.0)	2.88602 (5.0)	5.33109 (7.0)	2.82831 (4.0)

and p-value of $7.18e - 35$ show clear significant differences. The ranks can be seen in Table 3, while the average ranks are found in Table 6. A post-hoc analysis with the Nemenyi test, whose detailed results can be seen in Table 9, shows that LTSF-Linear models show significant pair-wise differences from most Transformer-based models, except FEDformer. However, looking at the best vs best comparison, NLinear's performance superiority over FEDformer is also significant. Henceforth, it is clear that the best LTSF-Linear model is significantly better than the best Transformer. Also note that the performance differences between FED-

former and Linear, and FEDformer and DLinear, are not significant.

The second and third steps are summarized in Table 4 and are applied independently to each dataset, considering the 10 training instances as the conditions of study as explained in Sect. 3.2.1. First, we apply the Friedman test, with the null hypothesis H_0 that all seven models are equally effective for $\alpha = 0.05$ and $df = 6$, whose critical value is 12.592, for which the average ranks can be found in Table 7. Since all of the computed test statistic values are above the critical value, we can reject H_0 and conclude that the differences are sig-

Table 4 Friedman test with post-hoc analysis based on the Nemenyi test. The first two columns display the statistic χ_F^2 and p-values of the tests applied to the 10 measurements of all the models (average ranks in Table 7). The second pair compares the 3 LTSF-Linear models with the best Transformer (average ranks in Table 8). Green cells denote significance at $\alpha = 0.05$ and blue at $\alpha = 0.1$. In all the experiments there are

significant performance differences, which also show significance by the Nemenyi test when we look at the best model of each family in half of the experiments when considering all models, and 78% of the experiments when including only the best Transformer in the comparison with the LTSF-Linear

Description		Friedman: all models included		Post-hoc Nemenyi: all models, pair-wise comparison. Pair shown: best LTSF - best Transformer	Friedman: all LTSF and only the best Transformer		Post-hoc Nemenyi: all LTSF and best Transformer. Pair shown: best LTSF - best Transformer
Metric		p-value	χ_F^2	p-value	p-value	χ_F^2	p-value
Electricity	96	$6.86e-11$	59.1	0.077	$4.67e-06$	27.48	0.001
	192	$8.21e-11$	58.71	0.058	$4.67e-06$	27.48	0.001
	336	$6.86e-11$	59.1	0.077	$4.67e-06$	27.48	0.001
	720	$1.44e-10$	57.51	0.129	$5.89e-06$	27.0	0.001
Exchange	96	$1.94e-10$	56.87	0.1	$9.39e-05$	21.24	0.001
	192	$3.9e-10$	55.37	0.626	0.000209	19.56	0.029
	336	$7.99e-10$	53.83	0.129	0.000415	18.12	0.002
	720	$5.58e-10$	54.6	0.129	0.000149	20.28	0.001
Traffic	96	$5.39e-11$	59.61	0.031	$1.38e-06$	30.0	0.001
	192	$9.26e-11$	58.46	0.023	$2.33e-06$	28.92	0.001
	336	$7.28e-11$	58.97	0.008	$1.38e-06$	30.0	0.001
	720	$7.73e-11$	58.84	0.023	$1.38e-06$	30.0	0.001
Weather	96	$1.2e-10$	57.9	0.023	$2.33e-06$	28.92	0.001
	192	$1.13e-10$	58.03	0.077	$1.18e-05$	25.56	0.001
	336	$1.5e-10$	57.43	0.016	$6.61e-06$	26.76	0.001
	720	$2.95e-10$	55.97	0.207	$9.39e-05$	21.24	0.001
ETTh1	96	$1.56e-10$	57.34	0.9	$1.18e-05$	25.56	0.226
	192	$3.02e-09$	50.96	0.687	0.056	7.56	0.046
	336	$1.54e-09$	52.41	0.565	0.0136	10.68	0.017
	720	$3.53e-10$	55.59	0.626	0.000176	19.92	0.029
ETTTh2	96	$3.82e-10$	55.41	0.9	$5.89e-06$	27.0	0.307
	192	$1.3e-10$	57.73	0.9	$5.56e-06$	27.12	0.307
	336	$8.89e-11$	58.54	0.9	$3.49e-06$	28.08	0.307
	720	$7.28e-11$	58.97	0.9	$5.56e-06$	27.12	0.307
ETTm1	96	$7.28e-11$	58.97	0.1	$5.56e-06$	27.12	0.001
	192	$1.04e-10$	58.2	0.077	$4.67e-06$	27.48	0.001
	336	$9.83e-11$	58.33	0.129	$1.32e-05$	25.32	0.001
	720	$2.57e-10$	56.27	0.165	$5.28e-05$	22.44	0.001
ETTm2	96	$6.81e-10$	54.17	0.31	0.000415	18.12	0.002
	192	$2.67e-10$	56.19	0.9	$3.49e-06$	28.08	0.307
	336	$9.44e-11$	58.41	0.9	$3.49e-06$	28.08	0.307
	720	$1.56e-10$	57.34	0.869	$4.67e-06$	27.48	0.307
ILL	24	$8.21e-11$	58.71	0.011	$2.33e-06$	28.92	0.001
	36	$6.86e-11$	59.1	0.031	$4.67e-06$	27.48	0.001
	48	$8.72e-11$	58.59	0.031	$5.56e-06$	27.12	0.001
	60	$1.56e-10$	57.34	0.005	$4.67e-06$	27.48	0.001

nificant with 95% confidence. This conclusion is reinforced by the p-values being below 0.05. For post-hoc analysis, the Nemenyi test offers a way to make pair-wise comparisons to detect whether measurements are significantly different, which can be extrapolated to the performance metrics of each model. To summarize these results, Table 4 shows the p-values of the pair best LTSF vs best Transformer. For $\alpha = 0.05$, the best LTSF-Linear model is significantly superior to the best Transformer on 10 occasions, while, for $\alpha = 0.1$, it happens for another 5 additional experiments. Thus, for 42% of the experiments, the best LTSF-Linear models are superior to the best Transformer for $\alpha = 0.1$.

In the third step, we refine the statistical comparisons by comparing the 3 LTSF-Linear models with the best Transformer for each dataset and prediction horizon. This can be useful to assess whether the best Transformer is significantly worse than the linear models without the interference of the much lower performance of the other Transformer-based models. For $\alpha = 0.05$ and $df = 3$ the critical value is 7.815, while for $\alpha = 0.1$ it is 6.251. The average ranks can be seen in Table 8. The best Transformer from the ones we tested is always the FEDformer, except for the Traffic dataset and $Z_p = 336$, where Autoformer performs slightly better, although both measurements are very similar. The fourth and fifth columns of Table 4 show the results of this set of comparisons, proving that there are significant differences between the LTSF-Linear models and the best Transformer by the Friedman test for all experiments for $\alpha = 0.05$, and for $\alpha = 0.1$ for the ETTh1 with $Z_p = 192$. The post-hoc analysis shows that the differences between the best LTSF-Linear and best Transformer are significant for $\alpha = 0.05$ in 28 out of 36 experiments, 78%. Also, note that, for the only case where FEDformer outperforms the best LTSF-Linear (DLinear), in dataset ETTh1 with $Z_p = 96$, the Nemenyi tests for $\alpha = 0.05$ and $\alpha = 0.1$ show that the differences are not significant.

Table 5 shows the σ of the performance metric analyzed. Despite the complexity of Transformer-based models and their high performance in other tasks, they are outperformed by the LTSF-Linear models. In addition to their performance issue already documented in the literature, Transformers also reveal significantly worse robustness to training stochasticity, as shown by higher σ_{MSE} for almost every dataset. The increase in the σ between Linear and Transformer-based models ranges up to 100%, with an average of 88.47%, giving further proof that Transformer-based models struggle to learn time dependencies from time series data that simpler models do.

Notably, there is one case in which the LTSF-Linear models are less robust than a Transformer in Table 5. For dataset ETTh2, FEDformer does a better job in terms of stability than the rest of the datasets, and with $Z_p = 192$ it can even surpass the NLinear model robustness. To provide insight into why

this happens, Fig. 4 shows some further information about ETTh1 and ETTh2. Both record the same data, but from different periods. The main contrast between the two datasets is the anomalous behavior shown in Fig. 4b, where some features spike to very different values, arguably displaying a different behavior. To the best of our knowledge, we did not find a reason for this anomaly, but its effects can also be seen in the autocorrelation and partial autocorrelation functions for both datasets, Fig. 4d and Fig. 4c, as the periodicity, although noticeable in both, is much clearer for ETTh1. This easy-to-pick pattern may explain why FEDformer can provide more robust forecasts for every but the longer prediction horizon, and its higher complexity allows it to account for this anomaly without impoverishing the predictions, based on the premise that neural networks are generally more robust to outliers than linear regression models. The more robust learning of NLinear for the longer prediction horizon may be similarly explained. While FEDformer can incorporate outliers (or anomalies in this case) more robustly into its training, it still performs generally worse for this dataset, as shown in Table 3. While for shorter prediction horizons FEDformer performs more robustly, for longer ones the linear model can compensate with more accurate temporal dependencies.

Although the average performance of LTSF-Linear models is still better, we can observe good performance on both. Nonetheless, they still perform worse than the simple NLinear models that it is compared against. Figure 5 compares the predictions made by these two models in this particular scenario, displaying visually the similarity of FEDformer and NLinear forecasts.

Figure 7a and Fig. 7b show the best-performing models, in contrast to the best averaged performance, for each dataset for the shortest and longest prediction horizons. In these figures, the direct comparison between the best models shows how the LTSF-Linear models achieve better performance with only one exception: FEDformer with the ETTh1 dataset. However, both best models achieve close MSE values, and the performance of FEDformer worsens for the long prediction horizon, where, at $Z_p = 720$, the difference in favor of the NLinear model is more noticeable. The error bars in Fig. 7 showcase that the LTSF-Linear models are generally more stable and robust in learning to forecast these time series. Interestingly, FEDformer keeps up with the linear models in robustness for shorter prediction horizons in ETTh1, ETTh2, electricity, and weather datasets, and even beats the DLinear and Linear models for the exchange rate dataset, both for shorter and longer prediction horizons. A plausible cause for this is the particular nature of the exchange dataset, which is further explored in Fig. 6. This dataset not present any periodicity or trend through the correlation and partial autocorrelation plots seen in Fig. 6b barely provide any useful insight aside from the fact that each time step is extremely highly correlated with the previous one and that the dynam-

Table 5 Standard deviation σ of the performance across 10 training instances where the best results are marked in green, best of each family in bold, and percentage of improvement under Imp. column. Red values indicate where the improvement percentage is in favor of Transformers. LTSF-Linear models consistently display more robust learning

than the Transformers except for the ETTh2 dataset, where FEDformer surpasses them for $Z_p = \{96, 192, 336\}$, although for $Z_p = 720$ the NLinear model is more robust and by a larger margin. FEDformer shows better robustness than other Transformer-based models, only scarcely surpassed by Autoformer in the Weather dataset

Model		Linear	NLinear	DLinear	Imp.	Transformer	Autoformer	Informer	FEDformer
Metric		σ_{MSE}	σ_{MSE}	σ_{MSE}	% σ_{MSE}	σ_{MSE}	σ_{MSE}	σ_{MSE}	σ_{MSE}
Electricity	96	0.00002	0.00003	0.00001	99.37%	0.00570	0.00359	0.00873	0.00164
	192	0.00002	0.00001	0.00002	99.84%	0.00524	0.00739	0.00741	0.00694
	336	0.00003	0.00001	0.00002	99.78%	0.00890	0.01447	0.01231	0.00384
	720	0.00014	0.00007	0.00014	98.64%	0.01066	0.04179	0.00966	0.00533
Exchange	96	0.01148	0.00111	0.00449	53.46%	0.10787	0.00672	0.03615	0.00238
	192	0.02134	0.00066	0.02031	91.54%	0.13668	0.06565	0.04585	0.00781
	336	0.01668	0.00126	0.01355	86.10%	0.21241	0.11141	0.05914	0.00905
	720	0.05847	0.00554	0.07428	75.66%	0.26288	0.15641	0.21122	0.02276
Traffic	96	0.00004	0.00014	0.00004	99.20%	0.00721	0.02016	0.01958	0.00454
	192	0.00013	0.00015	0.00007	98.92%	0.01225	0.02996	0.00883	0.00608
	336	0.00002	0.00009	0.00004	99.37%	0.00653	0.01154	0.02384	0.00346
	720	0.00005	0.00010	0.00004	99.24%	0.01026	0.01929	0.06723	0.00526
Weather	96	0.00044	0.00029	0.00029	98.75%	0.02880	0.02288	0.03816	0.02292
	192	0.00065	0.00013	0.00037	99.16%	0.05168	0.01508	0.03391	0.01965
	336	0.00058	0.00030	0.00094	97.99%	0.03965	0.01474	0.03755	0.08347
	720	0.00170	0.00019	0.00227	97.97%	0.08225	0.00931	0.07892	0.01042
ETTh1	96	0.00314	0.00155	0.00156	0.68%	0.08845	0.02358	0.08395	0.00156
	192	0.00143	0.00170	0.00251	50.79%	0.10004	0.02844	0.04530	0.00291
	336	0.00073	0.00492	0.00493	89.04%	0.07688	0.02896	0.04407	0.00670
	720	0.00353	0.00110	0.00223	92.34%	0.05119	0.01630	0.01552	0.01430
ETTh2	96	0.02001	0.00269	0.02382	0.50%	0.21535	0.01502	0.51673	0.00267
	192	0.03502	0.00523	0.03238	16.05%	0.99964	0.01585	0.38032	0.00439
	336	0.04831	0.00741	0.04898	0.92%	0.54660	0.01280	0.38874	0.00735
	720	0.05526	0.00482	0.07931	46.44%	0.37858	0.01292	0.25225	0.00900
ETTm1	96	0.00048	0.00318	0.00060	92.09%	0.02353	0.03872	0.02969	0.00604
	192	0.00084	0.00156	0.00040	95.95%	0.02389	0.02976	0.03119	0.00997
	336	0.00075	0.00110	0.00131	92.99%	0.05096	0.04624	0.04102	0.01069
	720	0.00190	0.00088	0.00114	92.43%	0.07059	0.07601	0.08750	0.01158
ETTm2	96	0.01118	0.00019	0.01021	87.43%	0.08602	0.04414	0.06990	0.00149
	192	0.01621	0.00050	0.01609	86.61%	0.11973	0.00532	0.13325	0.00374
	336	0.04129	0.00088	0.04761	77.39%	0.11446	0.01322	0.14559	0.00388
	720	0.08398	0.00046	0.04318	92.46%	0.44034	0.01344	0.51320	0.00610
ILI	24	0.05514	0.01170	0.11776	80.17%	0.17886	0.30455	0.09916	0.05898
	36	0.01652	0.03136	0.02348	63.97%	0.17273	0.11397	0.10300	0.04584
	48	0.05871	0.02754	0.04846	12.09%	0.14306	0.19095	0.09581	0.03133
	60	0.02933	0.01567	0.02437	55.71%	0.16616	0.13281	0.06486	0.03539

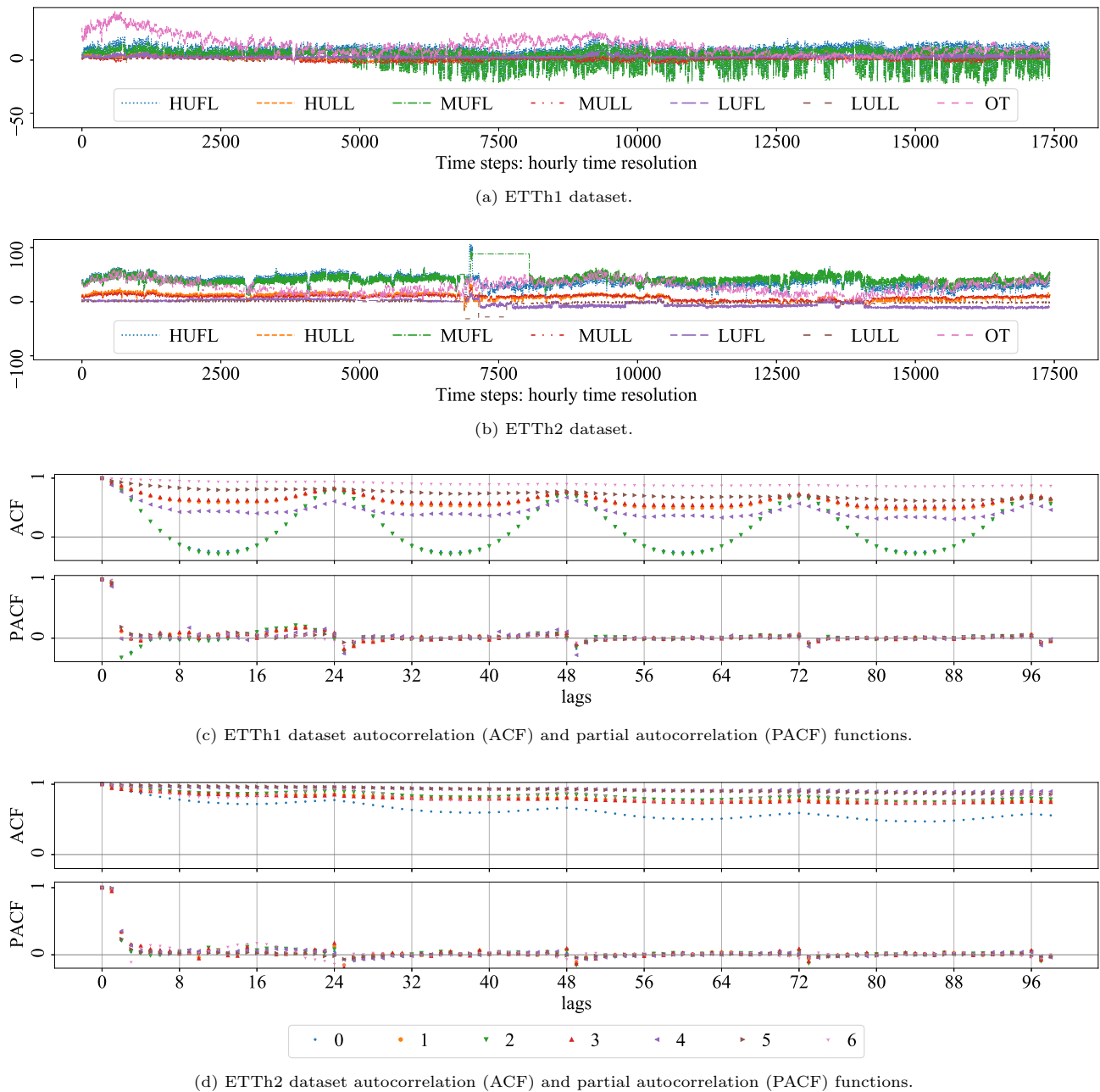


Fig. 4 ETTh1 and ETTh2 information. Both 4a and 4b show relatively fast dynamics in most of their features, possible periodicity based on the overlapping fast-changing signals, but no trends. An anomalous behavior affecting several features can be seen between time steps 6000 to 9000 in 4b that is not present in ETTh1. Both ACF for ETTh1 in 4c and

ETTh2 in 4d show that autocorrelation varies according to a 24-hour period, but it does so more steeply in ETTh1 than in ETTh2. PACF shows again similar characteristics, with the only difference that the spikes of PACF every 24-hour period are more significant in ETTh1. Note the first lag refers to lag 0, hence ACF and PACF values are 1

ics of this data are slow as seen in Fig. 6a, as befitting the currency exchange rates between countries.

Although the promising results of FEDformer suggest that Transformer-based models can compete with simpler models in time series forecasting tasks' performance and

robustness, the overall robustness difference points toward a higher uncertainty in Transformer-based models, which may be indicative of a faulty ability to learn the time dependencies consistently for most cases.

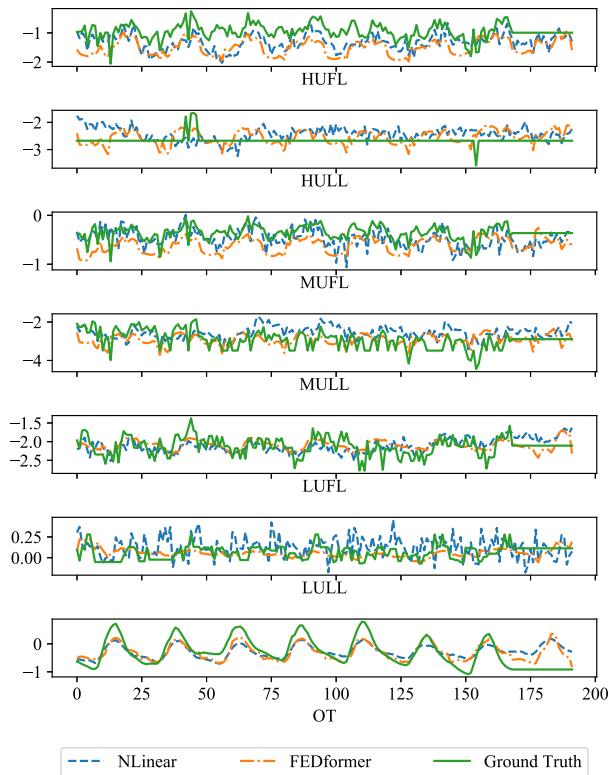


Fig. 5 Predictions of the best linear and Transformer-based model for a prediction horizon of $Z_p = 192$ for the ETTh2 dataset

5 Conclusion and future work

This work adds to the questioning of Transformers' suitability for time series forecasting tasks. We use the quite simple LTSF-Linear models from the literature in comparison with popular Transformer-based models to analyze robustness and stability differences, in addition to the already existing performance comparisons. Our results show an overall dramatic lack of stability and robustness from the Transformer-based models' side with respect to the LTSF-Linear models, which supports other literature conclusions that Transformers do not perform that well in time series forecasting tasks. This is reinforced by the results of the statistical tests, with the Friedman study combined with the Nemenyi post-hoc analysis showing significant differences in performance between models, and, on many experiments, with this significance being effective when comparing the best model of each family.

Facing the lack of robustness that the Transformer-based models have shown in this work, looking into them through eXplainable AI (XAI) lenses may provide a more accurate view of their inner workings. Using XAI methods has the potential to show what patterns the Transformer-based models have learned, revealing what time dependencies they have gathered after training, hence directly answering the question of whether they have discovered appropriate time dependencies for the task at hand or not.

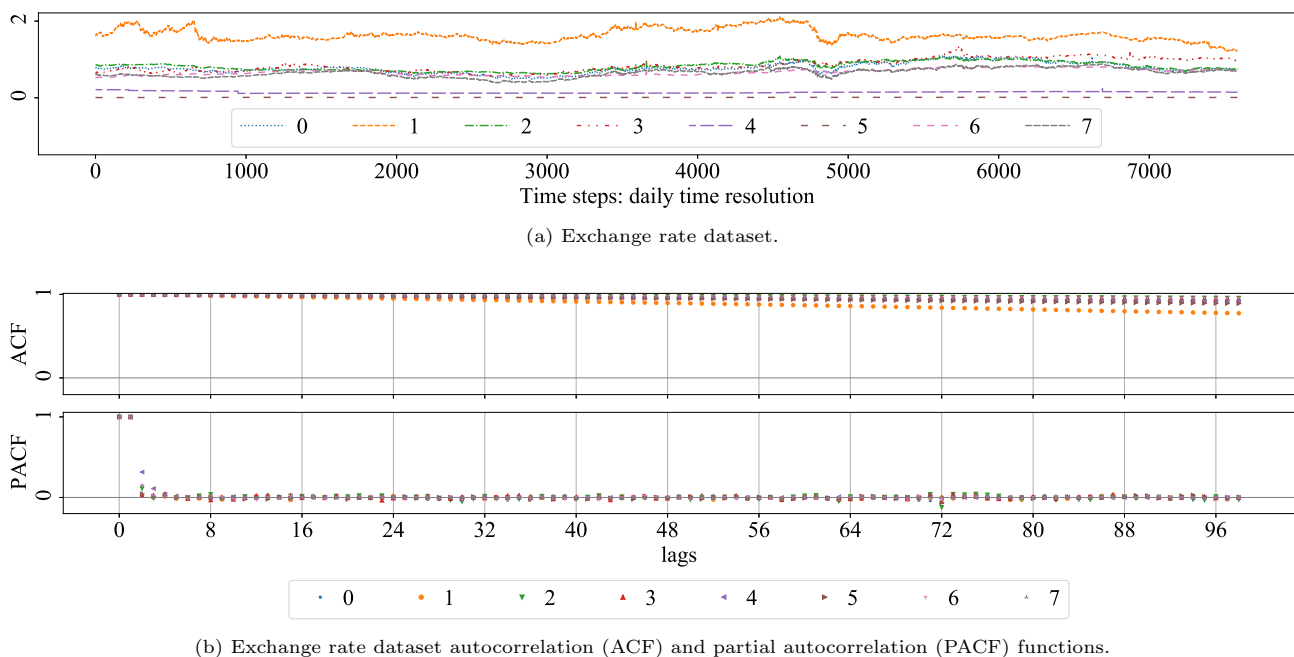


Fig. 6 Exchange rate dataset information. The dataset shows in 6a slow dynamics in all its features, with gradual changes over time, and no visible trends or periodicities. In 6b, ACF barely decreases, while PACF shows that the first lag holds most of the partial autocorrelation with

values near 1 for every feature, which drastically reduces in subsequent lags, becoming insignificant after the 3th lag. Note the first lag refers to lag 0, hence is always 1

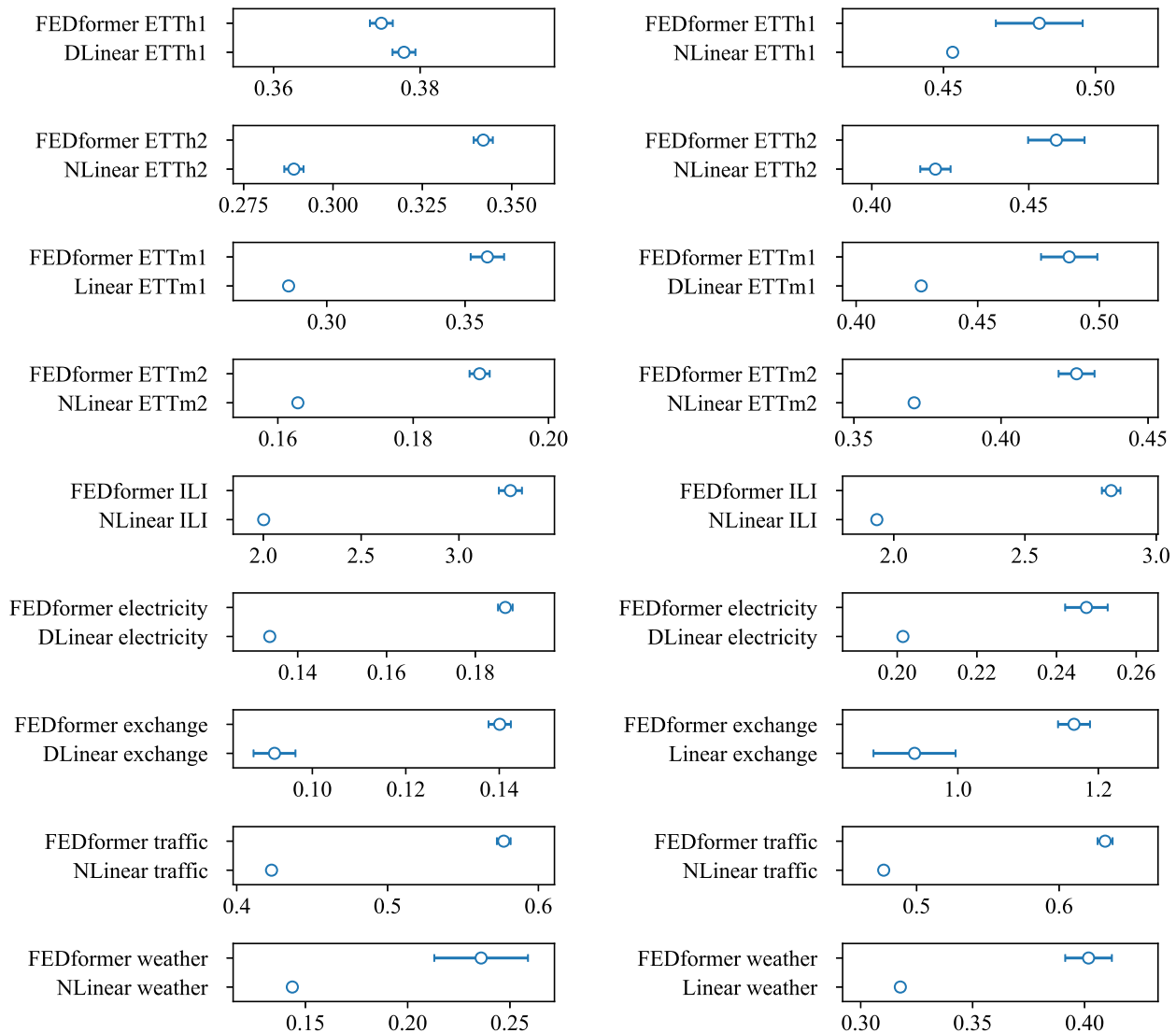


Fig. 7 Best LTSF and Transformer-based models for shorter and longer prediction horizons. The dots indicate the MSE, and the error bars display the σ . Note that the only instance where a Transformer-based model has more performance is the FEDformer with the ETTh1 dataset, but it worsens for $Z_p = 720$

Moreover, as research in Transformer-based model design and development progresses extremely quickly, keeping comparisons updated and valuable becomes challenging. Consequently, following our findings and those of other authors, future work on Transformers and time series forecasting would benefit from including simpler yet well-performing models in their experiments and result analysis. Considering not only performance but also robustness and statistical significance to compare their suitability for the task would help in keeping dependable comparisons that carry more weight to the research community.

Appendix A Complementary statistics results

This section shows the average rank used for the calculation of the Friedman and Nemenyi tests for the convenience of the reader. Table 6 shows the average ranks of the models for the global Friedman test applied considering all datasets and models, which are assigned based on the mean MSE of the 10 trained instances of each model. Note that all the average ranks of the LTSF-Linear models are lower than those of the Transformer-based models. The post-hoc Nemenyi results are shown in Table 9, and it can be seen that one or more

Table 6 Average ranks of the models considering all datasets and prediction horizons, where d refers to each dataset- Z_p combination. The average MSE across the 10 independent training instances of each modelfor each dataset- Z_p combination shown in Table 3 is used to assign the base ranks. The best model in each family is marked in bold, and the overall best is highlighted in green

Linear	NLinear	DLinear	Transformer	Autoformer	Informer	FEDformer
2.75	1.69	2.47	6.19	4.58	6.80	3.50

Table 7 Average model ranks per dataset Z_p combination, where d refers to the 10 independent training runs used in the per-dataset Friedman tests applied to all models, corresponding to the first results column

of Table 4. The best model in each family is marked in bold, and the overall best is highlighted in green

Model		Linear	NLinear	DLinear	Transformer	Autoformer	Informer	FEDformer
Electricity	96	1.7	3.0	1.3	6.0	5.0	7.0	4.0
	192	1.7	3.0	1.3	6.0	4.9	7.0	4.1
	336	1.7	3.0	1.3	6.0	5.0	7.0	4.0
	720	1.5	3.0	1.5	5.8	5.3	6.9	4.0
Exchange	96	2.3	2.3	1.4	6.0	5.0	7.0	4.0
	192	1.9	2.4	1.7	6.7	5.0	6.3	4.0
	336	2.1	2.0	1.9	6.7	4.6	6.3	4.4
	720	1.8	2.5	1.7	6.6	4.7	6.4	4.3
Traffic	96	2.0	1.0	3.0	5.9	5.1	7.0	4.0
	192	2.1	1.0	2.9	5.8	5.1	7.0	4.1
	336	2.0	1.0	3.0	6.0	4.4	7.0	4.6
	720	2.0	1.0	3.0	5.8	5.1	7.0	4.1
Weather	96	3.0	1.1	1.9	6.6	4.8	6.4	4.2
	192	2.9	1.4	1.7	7.0	4.9	6.0	4.1
	336	2.8	1.1	2.1	7.0	4.8	5.9	4.3
	720	1.7	2.6	1.7	6.3	5.0	6.7	4.0
ETTh1	96	3.3	3.6	2.1	6.5	5.0	6.5	1.0
	192	2.5	1.6	2.8	6.4	5.0	6.6	3.1
	336	2.3	1.5	3.0	6.2	5.0	6.8	3.2
	720	3.4	1.0	3.1	6.0	4.9	7.0	2.6
ETTh2	96	4.3	1.0	4.0	6.0	3.6	7.0	2.1
	192	4.6	1.0	4.3	6.2	3.1	6.8	2.0
	336	4.8	1.0	4.2	6.1	2.9	6.9	2.1
	720	4.4	1.0	4.6	6.0	3.0	7.0	2.0
ETTm1	96	1.4	3.0	1.6	6.0	5.0	7.0	4.0
	192	1.7	3.0	1.3	6.3	5.0	6.7	4.0
	336	1.5	2.9	1.6	6.0	5.0	7.0	4.0
	720	1.7	2.7	1.6	6.4	5.0	6.6	4.0
ETTm2	96	3.0	1.0	2.9	6.3	5.1	6.6	3.1
	192	4.6	1.0	3.7	6.8	3.7	6.2	2.0
	336	4.2	1.0	4.8	6.3	3.0	6.7	2.0
	720	4.3	1.0	4.7	6.5	2.8	6.5	2.2
ILI	24	2.9	1.0	2.1	6.0	4.7	7.0	4.3
	36	2.7	1.0	2.3	6.0	5.0	7.0	4.0
	48	2.6	1.0	2.5	6.1	5.0	6.9	4.0
	60	2.7	1.0	2.3	6.1	4.5	9.8	4.5

Table 8 Average model ranks per dataset Z_p combination for the Friedman tests applied to the three LTSF-Linear models and the best-performing Transformer, corresponding to the third results column of Table 4, where d refers to the 10 independent training runs. The best model in each family is marked in bold, and the overall best is highlighted in green

Model		Linear	NLinear	DLinear	Best Transformer
Electricity	96	1.7	3.0	1.3	(F) 4.0
	192	1.7	3.0	1.3	(F) 4.0
	336	1.7	3.0	1.3	(F) 4.0
	720	1.5	3.0	1.5	(F) 4.0
Exchange	96	2.3	2.3	1.4	(F) 4.0
	192	1.9	2.4	1.7	(F) 4.0
	336	2.1	2.0	1.9	(F) 4.0
	720	1.8	2.5	1.7	(F) 4.0
Traffic	96	2.0	1.0	3.0	(F) 4.0
	192	2.1	1.0	2.9	(F) 4.0
	336	2.0	1.0	3.0	(A) 4.0
	720	2.0	1.0	3.0	(F) 4.0
Weather	96	3.0	1.1	1.9	(F) 4.0
	192	2.9	1.4	1.7	(F) 4.0
	336	2.8	1.1	2.1	(F) 4.0
	720	1.7	2.6	1.7	(F) 4.0
ETTh1	96	3.3	3.6	2.1	(F) 1.0
	192	2.5	1.6	2.8	(F) 3.1
	336	2.3	1.5	3.0	(F) 3.2
	720	3.4	1.0	3.0	(F) 2.6
ETTh2	96	3.5	1.0	3.5	(F) 2.0
	192	3.6	1.0	3.4	(F) 2.0
	336	3.8	1.0	3.2	(F) 2.0
	720	3.4	1.0	3.6	(F) 2.0
ETTm1	96	1.4	3.0	1.6	(F) 4.0
	192	1.7	3.0	1.3	(F) 4.0
	336	1.5	2.9	1.6	(F) 4.0
	720	1.7	2.7	1.6	(F) 4.0
ETTm2	96	3.0	1.0	2.9	(F) 3.1
	192	3.8	1.0	3.2	(F) 2.0
	336	3.2	1.0	3.8	(F) 2.0
	720	3.3	1.0	3.7	(F) 2.0
ILI	24	2.9	1.0	2.1	(F) 4.0
	36	2.7	1.0	2.3	(F) 4.0
	48	2.6	1.0	2.4	(F) 4.0
	60	2.7	1.0	2.1	(F) 4.0

of the LTSF-Linear models significantly outperform the best Transformer in all pair-wise comparisons.

Table 7 and Table 8 show the average ranks for the Friedman test individually to each dataset and Z_p following the approaches described in Section 3.2.1, the first to all models, and the second to all LTSF-Linear and only the best Transformer-based model for each experiment. Note that for these tests, d refers to each of the training instances, not to the datasets. As expected, LTSF-Linear models are superior in average ranks in all cases except for the one where FEDformer outperforms them, shown in Table 3.

Table 9 P-values of the Nemenyi post-hoc test for the global comparison across all models and datasets, following the results of Table 6. Significance at $\alpha = 0.05$ is indicated in bold

	Linear	NLinear	DLinear	Transformer	Autoformer	Informer	FEDformer
Linear	1.000000	0.369901	0.900000	0.001000	0.005873	0.001000	0.733374
NLinear	0.369901	1.000000	0.701291	0.001000	0.001000	0.001000	0.007162
DLinear	0.900000	0.701291	1.000000	0.001000	0.001000	0.001000	0.404806
Transformer	0.001000	0.001000	0.001000	1.000000	0.026070	0.893765	0.001000
Autoformer	0.005873	0.001000	0.001000	0.026070	1.000000	0.001000	0.336257
Informer	0.001000	0.001000	0.001000	0.893765	0.001000	1.000000	0.001000
FEDformer	0.733374	0.007162	0.404806	0.001000	0.336257	0.001000	1.000000

Funding Open access funding provided by NTNU Norwegian University of Science and Technology (incl St. Olavs Hospital - Trondheim University Hospital) This work is funded by the Research Council of Norway through the EXAIGON project (project no. 304843).

Data Availability The code is available in our https://github.com/AlbertoMC126/ChronoSHAP_Transformers_LTSF-Linear_robustness and the datasets can be found in https://drive.google.com/drive/folders/1ZOYpTUa82_jCcxIdTmyr0LXQfvaM9vly.

Declarations

Conflicts of Interest The authors declare that there is no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Mahalakshmi, G., Sridevi, S., Rajaram, S.: A survey on forecasting of time series data. In: 2016 International Conference on Computing Technologies and Intelligent Data Engineering (ICCTIDE'16), pp. 1–8. (2016). <https://doi.org/10.1109/ICCTIDE.2016.7725358>
2. Mudelsee, M.: Climate Time Series Analysis. Classical Statistical and Bootstrap Methods., Second edition edn. Atmospheric and Oceanographic Sciences Library, vol. 51. Springer, Germany (2014). <https://doi.org/10.1007/978-3-319-04450-7>
3. Topol, E.J.: High-performance medicine: the convergence of human and artificial intelligence. *Nat. Med.* **25**(1), 44–56 (2019). <https://doi.org/10.1038/s41591-018-0300-7>
4. Lim, B., Zohren, S.: Time-series forecasting with deep learning: a survey. *Philosophical Trans. of the Royal Soc. A: Math., Phys. and Eng. Sci.* **379**(2194), 20200209 (2021). doi: 10.1098/rsta.2020.0209
5. Feyrer, J.: Trade and income-exploiting time series in geography. *Am. Econ. J. Appl. Econ.* **11**(4), 1–35 (2019). <https://doi.org/10.1257/app.20170616>
6. Makridakis, S., Hibon, M.: ARMA Models and the box Jenkins methodology. *J. Forecast.* **16**(3), 147–163 (1997). [https://doi.org/10.1002/\(SICI\)1099-131X\(199705\)16:3<147::AID-FOR652>3.0.CO;2-X](https://doi.org/10.1002/(SICI)1099-131X(199705)16:3<147::AID-FOR652>3.0.CO;2-X)
7. Harvey, A.C.: ARIMA models. In: Eatwell, J., Milgate, M., Newman, P. (eds.) *Time Series and Statistics*, pp. 22–24. Palgrave Macmillan UK, London (1990). https://doi.org/10.1007/978-1-349-20865-4_2
8. Lau, K.W., Wu, Q.H.: Local prediction of non-linear time series using support vector regression. *Pattern Recogn.* **41**(5), 1539–1547 (2008). <https://doi.org/10.1016/j.patcog.2007.08.013>
9. Gershman, S.J., Blei, D.M.: A tutorial on Bayesian nonparametric models. *J. Math. Psychol.* **56**(1), 1–12 (2012). <https://doi.org/10.1016/j.jmp.2011.08.004>
10. Fantozzi, P., Naldi, M.: The explainability of transformers: current status and directions. *Comput.* **13**(4), 92 (2024). <https://doi.org/10.3390/computers13040092>
11. Hornik, K.: Approximation capabilities of multilayer feedforward networks. *Neural Netw.* **4**(2), 251–257 (1991). [https://doi.org/10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T)
12. Lu, Y., Lu, J.: A universal approximation theorem of deep neural networks for expressing probability distributions. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 3094–3105. Curran Associates Inc, Canada (2020). <https://doi.org/10.48550/arXiv.2004.08867>
13. Calder, J., Coil, R., Melton, J.A., Olver, P.J., Tostevin, G., Yezzi-Woodley, K.: Use and misuse of machine learning in anthropology. *IEEE BITS the Inform. Theory Magazine* **2**(1), 102–115 (2022). <https://doi.org/10.1109/MBITS.2022.3205143>
14. Wei, M., Harzevili, N.S., Huang, Y., Yang, J., Wang, J., Wang, S.: Demystifying and detecting misuses of deep learning APIs. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, pp. 1–12. Association for Computing Machinery, New York, NY, USA (2024). doi: 10.1145/3597503.3639177. Accessed 2024-05-13
15. Kolarik, T., Rudorfer, G.: Time series forecasting using neural networks. *ACM SIGAPL APL Quote Quad* **25**(1), 86–94 (1994). <https://doi.org/10.1145/190468.190290>
16. Zhao, B., Lu, H., Chen, S., Liu, J., Wu, D.: Convolutional neural networks for time series classification. *J. Syst. Eng. Electron.* **28**(1), 162–169 (2017). <https://doi.org/10.21629/JSEE.2017.01.18>
17. Hewamalage, H., Bergmeir, C., Bandara, K.: Recurrent neural networks for time series forecasting: current status and future directions. *Int. J. Forecast.* **37**(1), 388–427 (2021). <https://doi.org/10.1016/j.ijforecast.2020.06.008>
18. Sirisha, U.M., Belavagi, M.C., Attigeri, G.: Profit prediction using ARIMA, SARIMA and LSTM models in time series forecasting: a comparison. *IEEE Access* **10**, 124715–124727 (2022). <https://doi.org/10.1109/ACCESS.2022.3224938>
19. Vaswani, A., Shazeer, N.M., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30, pp. 11. Curran Associates Inc, USA (2017). <https://arxiv.org/abs/1706.03762>. Accessed 2024-03-14
20. Dong, L., Xu, S., Xu, B.: Speech-transformer: a no-recurrence sequence-to-sequence model for speech recognition. In: 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 5884–5888 (2018). doi: 10.1109/ICASSP.2018.8462506. ISSN: 2379-190X
21. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North*, pp. 4171–4186. Association for Computational Linguistics, Minneapolis, Minnesota (2019). <https://doi.org/10.18653/v1/N19-1423>
22. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (2021). <https://doi.org/10.48550/arXiv.2010.11929>. arXiv:2010.11929 [cs]
23. Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J., Sun, L.: Transformers in Time Series: A Survey. [arXiv:2202.07125](https://arxiv.org/abs/2202.07125) [cs, eess, stat]. <https://arxiv.org/abs/2202.07125> (2023). Accessed 2024-04-09
24. Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., Zhang, W.: Informer: beyond efficient transformer for long sequence time-series forecasting. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11106–11115 (2021). doi: 10.1609/aaai.v35i12.17325. ISSN: 2374-3468, 2159-5399 Issue: 12 Journal Abbreviation: AAAI

25. Wu, H., Xu, J., Wang, J., Long, M.: Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. <https://doi.org/10.48550/arXiv.2106.13008> [cs] (2022)
26. Woo, G., Liu, C., Sahoo, D., Kumar, A., Hoi, S.: ETSformer: Exponential Smoothing Transformers for Time-series Forecasting. <https://doi.org/10.48550/arXiv.2202.01381>. arXiv:2202.01381 [cs] (2022)
27. Zhang, Y., Yan, J.: Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting. In: ICLR Proceedings 2023, pp. 21. <https://openreview.net/forum?id=vSVLM2j9eie> (2022). Accessed 2024-04-09
28. Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., Jin, R.: FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting. In: Proceedings of the 39th International Conference on Machine Learning, pp. 27268–27286. PMLR, USA (2022). ISSN: 2640–3498
29. Woo, G., Liu, C., Kumar, A., Xiong, C., Savarese, S., Sahoo, D.: Unified training of universal time series forecasting transformers. Proceedings of the 41st International Conference on Machine Learning (ICML 2024): Vienna, Austria, July 21–27 235, pp. 53140–53164 (2024)
30. Wang, Y., Wu, H., Dong, J., Qin, G., Zhang, H., Liu, Y., Qiu, Y., Wang, J., Long, M.: TimeXer: empowering transformers for time series forecasting with exogenous variables. Adv. Neural. Inf. Process. Syst. **37**, 469–498 (2025)
31. Liu, L., Liu, X., Gao, J., Chen, W., Han, J.: Understanding the Difficulty of Training Transformers. <https://doi.org/10.48550/arXiv.2004.08249>. arXiv:2004.08249 [cs] (2023)
32. Li, B., Huang, W., Han, A., Zhou, Z., Suzuki, T., Zhu, J., Chen, J.: On the Optimization and Generalization of Two-layer Transformers with Sign Gradient Descent. doi: <https://doi.org/10.48550/arXiv.2410.04870>. arXiv:2410.04870 [cs] (2024)
33. Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the Thirteenth International Conference on Artificial Intelligence And Statistics, JMLR Workshop and Conference Proceedings, pp. 249–256. Italy (2010). ISSN: 1938–7228. <https://proceedings.mlr.press/v9/glorot10a.html>. Accessed 2025-02-21
34. Frankle, J., Carbin, M.: The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. arXiv. arXiv:1803.03635 [cs]. <http://arxiv.org/abs/1803.03635> (2019) Accessed 2023-02-21
35. Frankle, J., Dziugaite, G.K., Roy, D.M., Carbin, M.: Linear Mode Connectivity and the Lottery Ticket Hypothesis. <https://doi.org/10.48550/arXiv.1912.05671>. arXiv:1912.05671 [cs, stat] (2020)
36. Zhang, S., Wang, M., Liu, S., Chen, P.-Y., Xiong, J.: Why Lottery Ticket Wins? A Theoretical Perspective of Sample Complexity on Pruned Neural Networks, (2021). (cs, math)
37. Taylor, S.J., Letham, B.: Forecasting at scale. Technical Report e3190v2, PeerJ Inc. <https://peerj.com/preprints/3190/> (September 2017). ISSN: 2167–9843. Accessed 2024-05-13
38. Amin, M.A.A., Hoque, M.A.: Comparison of ARIMA and SVM for Short-term Load Forecasting. In: 2019 9th Annual Information Technology, Electromechanical Engineering and Microelectronics Conference (IEMECON), pp. 1–6 (2019). doi: 10.1109/IEMECONX.2019.8877077
39. Karthika, S., Margaret, V., Balaraman, K.: Hybrid short term load forecasting using ARIMA-SVM. In: 2017 Innovations in Power and Advanced Computing Technologies (i-PACT), pp. 1–7. (2017). <https://doi.org/10.1109/IPACT.2017.8245060>
40. Zeng, A., Chen, M., Zhang, L., Xu, Q.: Are transformers effective for time series forecasting? Proc. of the AAAI Conf. on Artif. Intell. **37**(9), 11121–11128 (2023). doi: 10.1609/aaai.v37i9.26317
41. Kelley, H.J.: Gradient theory of optimal flight paths. ARS J. **30**(10), 947–954 (1960). <https://doi.org/10.2514/8.5282>
42. Li, S., Jin, X., Xuan, Y., Zhou, X., Chen, W., Wang, Y.-X., Yan, X.: Enhancing the Locality and Breaking the Memory Bottleneck of Transformer on Time Series Forecasting. <https://doi.org/10.48550/arXiv.1907.00235>. arXiv:1907.00235 [cs] (2020)
43. Clevert, D.-A., Unterthiner, T., Hochreiter, S.: Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). doi: <https://doi.org/10.48550/arXiv.1511.07289>. arXiv:1511.07289 [cs] (2016)
44. Swirszcz, G., Czarnecki, W.M., Pascanu, R.: Local minima in training of neural networks. <https://doi.org/10.48550/arXiv.1611.06310>. arXiv:1611.06310 [cs, stat] (2017)
45. Carlini, N., Wagner, D.: Towards Evaluating the Robustness of Neural Networks. In: 2017 IEEE Symposium on Security and Privacy (SP), pp. 39–57 (2017). doi: 10.1109/SP.2017.49. ISSN: 2375-1207
46. Li, Y.: Towards practical robustness evaluation and robustness enhancing. In: WSDM 2022 - Proceedings of the 15th ACM International Conference on Web Search And Data Mining, pp. 1549–1550. Association for Computing Machinery, USA (2022). <https://doi.org/10.1145/3488560.3502216>
47. Lamb, A., Verma, V., Kawaguchi, K., Matyasko, A., Khosla, S., Kannala, J., Bengio, Y.: Interpolated adversarial training: achieving robust neural networks without sacrificing too much accuracy. Neural Netw. **154**, 218–233 (2022). <https://doi.org/10.1016/j.neunet.2022.07.012>
48. Barber, F., Salido, M.A.: Robustness, stability, recoverability, and reliability in constraint satisfaction problems. Knowl. Inf. Syst. **44**(3), 719–734 (2015). <https://doi.org/10.1007/s11015-014-0778-3>
49. Freiesleben, T., Grote, T.: Beyond generalization: a theory of robustness in machine learning. Synthese **202**(4), 109 (2023). <https://doi.org/10.1007/s11229-023-04334-9>
50. Diehl, P.U., Neil, D., Binas, J., Cook, M., Liu, S.-C., Pfeiffer, M.: Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In: 2015 International Joint Conference on Neural Networks (IJCNN), pp. 1–8 (2015). doi: 10.1109/IJCNN.2015.7280696. ISSN: 2161-4407
51. Patrini, G., Rozza, A., Menon, A.K., Nock, R., Qu, L.: Making deep neural networks robust to label noise: A loss correction approach. In: Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, vol. 2017-January, pp. 2233–2241 (2017). doi: 10.1109/CVPR.2017.240
52. Springenberg, M., Frommholz, A., Wenzel, M., Weicken, E., Ma, J., Strodthoff, N.: From modern CNNs to vision transformers: assessing the performance, robustness, and classification strategies of deep learning models in histopathology. Med. Image Anal. **87**, 102809 (2023). <https://doi.org/10.1016/j.media.2023.102809>
53. Elisseeff, A., Evgeniou, T., Pontil, M.: Stability of randomized learning algorithms. J. Mach. Learn. Res. **6**(3), 55–79 (2005)
54. Bousquet, O., Elisseeff, A.: Stability and generalization. J. Mach. Learn. Res. **2**, 499–526 (2002)
55. Demšar, J.: Statistical comparisons of classifiers over multiple data sets. J. Mach. Learn. Res. **7**, 1–30 (2006)
56. Friedman, M.: The use of ranks to avoid the assumption of normality implicit in the analysis of variance. J. Am. Stat. Assoc. **32**(200), 675–701 (1937). <https://doi.org/10.2307/2279372>
57. Friedman, M.: A comparison of alternative tests of significance for the problem of m rankings. Ann. Math. Stat. **11**(1), 86–92 (1940)
58. Wong, C.K., Coleman, D., diPersia, V., Song, J., Wright, D.: The effects of manual treatment on rounded-shoulder posture, and associated muscle strength. J. Bodyw. Mov. Ther. **14**(4), 326–333 (2010). <https://doi.org/10.1016/j.jbmt.2009.05.001>
59. Bliddal, H., Rosetzsky, A., Schlichting, P., Weidner, M.S., Andersen, L.A., Ibfelt, H.H., Christensen, K., Jensen, O.N., Barslev, J.: A randomized, placebo-controlled, cross-over study of ginger

- extracts and ibuprofen in osteoarthritis. *Osteoarthritis Cartilage* 8(1), 9–12 (2000). <https://doi.org/10.1053/joca.1999.0264>
60. Iman, R.L., Davenport, J.M.: Approximations of the critical region of the Friedman statistic. *Commun. in Stat. - Theory and Methods* 9(6), 571–595 (1980). <https://doi.org/10.1080/03610928008827904>
 61. Nemenyi, P.B.: *Distribution-Free Multiple Comparisons*. Ph.D., Princeton University, United States - New Jersey. ISBN: 9781084733008 Publication Title: ProQuest Dissertations and Theses. <https://www.proquest.com/docview/302256074/citation/B8CA63CFCE29420APQ/1> (1963). Accessed 2025-06-11
 62. Lai, G., Chang, W.-C., Yang, Y., Liu, H.: Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks. In: *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval. SIGIR '18*, pp. 95–104. Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3209978.3210006>
 63. California Department of Transportation: Caltrans Performance Measurement System. <https://pems.dot.ca.gov/> (2002). Accessed 2024–05-13
 64. Trindade, A.: *ElectricityLoadDiagrams20112014*. [Dataset] (2015). <https://doi.org/10.24432/C58C86>
 65. Guokun Lai: GitHub - laiguokun/multivariate-time-series-data. <https://github.com/laiguokun/multivariate-time-series-data/tree/master> (2017). Accessed 2024–05-13
 66. Max Planck Institute for Biogeochemistry: Weather data from the Max-Planck-Institute for Biogeochemistry. <https://www.bgc-jena.mpg.de/wetter/> (2020). Accessed 2024–05-13
 67. Center for Disease Control and Prevention: National, Regional, and State Level Outpatient Illness and Viral Surveillance. <https://gis.cdc.gov/grasp/fluview/fluportaldashboard.html> (2017). Accessed 2024–05-13

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.